

优化扩展大语言模型测试时计算可能比扩展模型参数更有效

Charlie Snell^{*, 1}, Jaehoon Lee², Kelvin Xu^{*, 2} and Aviral Kumar^{*, 2}

^{*}Equal advising, ¹UC Berkeley, ²Google DeepMind, ^{*}Work done during an internship at Google DeepMind

使大语言模型（LLM）能够通过使用更多测试时计算来改进其输出，是构建能够在开放式自然语言上运行的一般自我改进智能体的关键一步。本文研究了大语言模型中推理时计算的扩展，重点回答以下问题：如果允许大语言模型使用固定但非平凡的推理时计算量，它在具有挑战性的提示上能提升多少性能？回答这个问题不仅对大语言模型可实现的性能有影响，而且对大语言模型预训练的未来以及如何权衡推理时计算和预训练计算也有影响。尽管这个问题很重要，但很少有研究试图理解各种测试时推理方法的扩展行为。此外，当前的工作在很大程度上为这些策略中的许多提供了负面结果。在这项工作中，我们分析了扩展测试时计算的两种主要机制：（1）针对密集的、基于过程的验证器奖励模型进行搜索；（2）在测试时给定提示的情况下，自适应地更新模型对响应的分布。我们发现，在这两种情况下，不同扩展测试时计算方法的效果关键取决于提示的难度。这一观察结果促使我们采用“计算最优”的扩展策略，该策略旨在为每个提示最有效地自适应分配测试时计算。使用这种计算最优策略，与最佳 N 基线相比，我们可以将测试时计算扩展的效率提高 $4 \times$ 倍以上。此外，在浮点运算次数匹配的评估中，我们发现在较小的基础模型取得一定非平凡成功率的难题上，测试时计算可以用来超越 $14 \times$ 倍的模型。

1. 引言

人类倾向于在困难问题上思考更长时间，以可靠地改进他们的决策 [9, 17, 18]。我们能否将类似的能力赋予当今的大语言模型（LLMs）？更具体地说，给定一个具有挑战性的输入查询，我们能否使语言模型在测试时最有效地利用额外的计算，以提高其响应的准确率？理论上，通过在测试时应用额外的计算，大语言模型应该能够比其训练时的表现更好。此外，这种测试时的能力还有可能为智能体和推理任务开辟新的途径 [28, 34, 47]。例如，如果预训练模型的大小可以在推理过程中用额外的计算来交换，这将使大语言模型能够在设备端使用较小模型替代数据中心规模的大语言模型的场景中部署。通过使用额外的推理时计算来自动生成改进的模型输出，也为一种通用的自我改进算法提供了一条路径，该算法可以在减少人类监督的情况下运行。先前研究推理时计算的工作给出了混合的结果。一方面，一些工作表明当前的大语言模型可以利用测试时计算来改进其输出 [4, 8, 23, 30, 48]，另一方面，其他工作表明这些方法在更复杂的任务（如数学推理）上的有效性仍然非常有限 [15, 37, 43]，尽管推理问题通常需要对现有知识进行推断，而不是获取新知识。这些相互矛盾的发现促使我们需要对不同扩展测试时计算的方法进行系统分析。

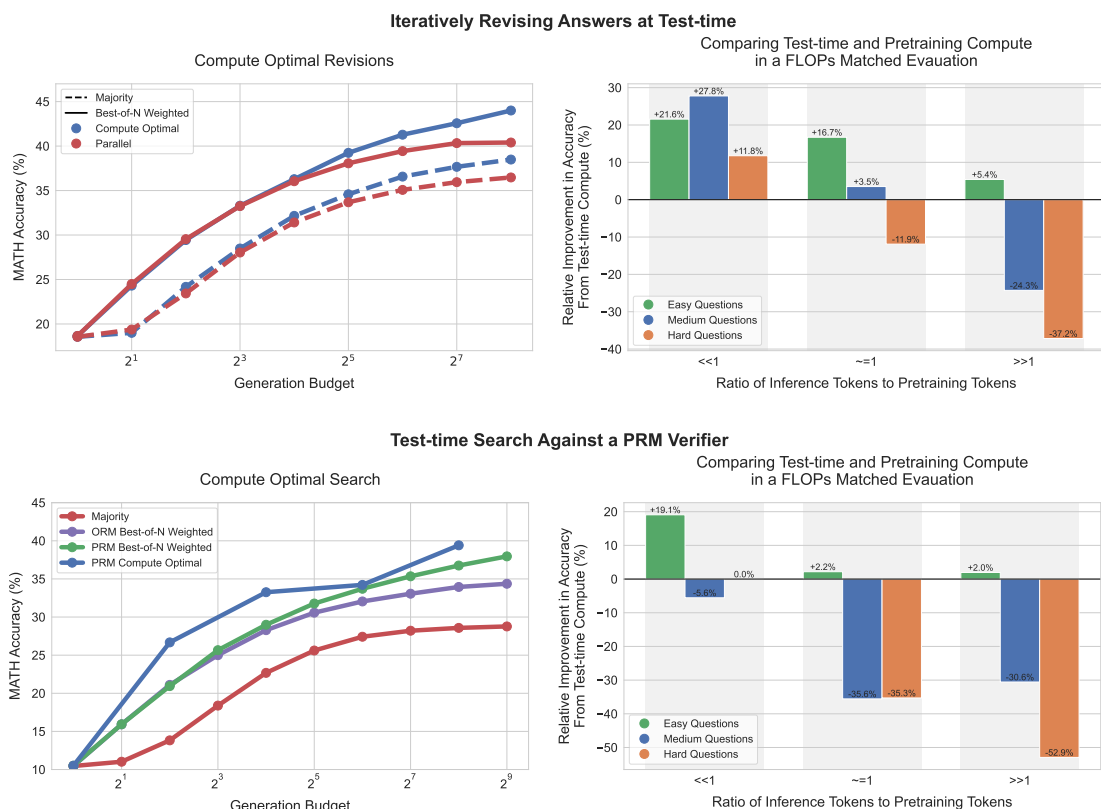


图 1 | 主要结果总结。左图：迭代自精炼（即修订）与搜索的计算最优扩展。左侧，我们比较了 PaLM 2-S* 修订模型的计算最优扩展策略与修订设置（上）和 PRM 搜索设置（下）中的基线。我们看到在修订情况下，标准最佳 N 选（例如“并行”）与计算最优扩展之间的差距逐渐扩大，使计算最优扩展能够以少 $4 \times$ 的测试时计算超越最佳 N 选。类似地，在 PRM 搜索设置中，我们观察到计算最优扩展相对于最佳 N 选在早期就有显著改进，在某些点上几乎以少 $4 \times$ 的计算超越最佳 N 选。详见第 5 节和第 6 节。右图：测试时计算与模型参数扩展的比较。我们将 PaLM 2-S* 的计算最优测试时扩展性能与一个 $\sim 14 \times$ 更大的预训练模型在没有额外测试时计算（例如贪心采样）下的性能进行比较。我们考虑这样的设置：两个模型都预期有 x 的预训练 token 和 y 的推理 token。通过训练更大的模型，我们实际上成倍增加了这两项所需的 FLOPs。如果我们较小的模型应用额外的测试时计算，以匹配这个更大模型的 FLOPs 需求，那么在准确率方面会如何比较？我们看到对于修订（上）当 $y \ll x$ 时，测试时计算通常优于额外的预训练。然而，随着推理与预训练 token 比率增加，测试时计算在简单问题上仍然更优。而在更难的问题上，预训练在这些设置中更优。我们在 PRM 搜索（下）中也看到了类似的趋势。详见第 7 节。

我们旨在理解扩展测试时计算所带来的益处。最直接且研究最为深入的测试时计算扩展方法当属 N 中择优采样：从基础大语言模型（LLM）中“并行”采样 N 个输出，并根据学习到的验证器或奖励模型 [7, 22] 选择得分最高的一个。然而，这并非利用测试时计算改进大语言模型的唯一途径。通过修改获取响应的提议分布（例如，要求基础模型“顺序地”修订其原始响应 [28]），或改变验证器的使用方式（例如，训练基于过程的密集验证器 [22, 45] 并针对该验证器进行搜索），可以显著提升扩展测试时计算的能力，正如本文所示。为理解扩展测试时计算的益处，我们在具有挑战性的数学（MATH）[13] 基准上，使用经过特定微调¹的 PaLM-2[3] 模型进行了实验，使其能够修订 1 在数学基准上，针对能力的特定微调对于在基础模型中引入修订和验证能力是必要的。

错误答案 [28]（例如改进提议分布；第 6 节）或使用基于过程的奖励模型（PRM）[22, 45] 验证答案中各个步骤的正确性（第 5 节）。通过这两种方法，我们发现特定测试时计算策略的有效性关键取决于手头具体问题的性质以及所使用的基础大语言模型。例如，对于较简单的问题，基础大语言模型已经能够轻松生成合理的回答，此时允许模型通过预测一系列 N 次修订（即修改提议分布）来迭代改进其初始答案，可能比并行采样 N 个独立回答更有效地利用测试时计算。另一方面，对于可能需要搜索多种不同高层解题方法的更困难问题，独立并行重新采样新回答或针对基于过程的奖励模型部署树搜索，可能是更有效的测试时计算利用方式。这一发现表明，需要部署一种自适应的“计算最优”策略来扩展测试时计算，其中根据提示选择具体的测试时计算利用方法，以充分利用额外计算。我们还表明，从基础大语言模型的角度来看，问题难度（第 4 节）的概念可用于预测测试时计算的有效性，使我们能够在给定提示的情况下实际实例化这种“计算最优”策略。通过以这种方式适当分配测试时计算，我们能够大幅改善测试时计算的扩展性，在仅使用约 4 倍更少计算量的情况下，通过修订和搜索（第 5 节和第 6 节）超越了 N 选一基线的性能。利用我们改进的测试时计算扩展策略，我们进而旨在理解测试时计算在多大程度上能够有效替代额外的预训练。我们进行了 FLOPs 匹配的比较，将具有额外测试时计算的较小模型与预训练大 14 倍的模型进行对比。我们发现，在简单和中等难度的问题上，甚至在困难问题上（取决于预训练和推理工作负载的具体条件），额外的测试时计算通常比扩展预训练更可取。这一发现表明，与其纯粹专注于扩展预训练，在某些情况下，用更少的计算量预训练较小的模型，然后应用测试时计算来改进模型输出，可能更为有效。尽管如此，对于最具挑战性的问题，我们观察到扩展测试时计算的收益甚微。相反，我们发现对于这些问题，通过应用额外的预训练计算来取得进展更为有效，这表明当前扩展测试时计算的方法可能无法与扩展预训练进行一对一的互换。总体而言，这表明即使采用相当朴素的方法论，扩展测试时计算已经可以比扩展预训练更可取，并且随着测试时策略的成熟，还将获得更多改进。从长远来看，这预示着未来在预训练阶段花费更少的 FLOPs，而在推理阶段花费更多的 FLOPs。

2. 测试时计算的统一视角：提议器与验证器

本文首先统一了利用测试时计算的方法，随后分析若干代表性方法。首先，本文从在测试时根据给定提示自适应地修改模型预测分布的角度，审视额外测试时计算的使用。理想情况下，测试时计算应修改分布，以生成比直接从大语言模型（LLM）朴素采样更优的输出。一般而言，诱导大语言模型分布修改有两个旋钮：（1）在输入层面：

因为即使在强大的专有大语言模型中，这些能力也是缺失的 [15, 33]。然而，本文预期未来的大语言模型将因规模扩大以及纳入专门针对这些能力的额外数据，而在验证与修订方面更为有效 [5, 24, 36]。因此，为了推进对测试时计算扩展的理解，必须使用针对这些能力进行微调的模型。尽管如此，本文预期未来模型将直接针对此类能力进行预训练，从而避免针对特定能力的微调需求。

通过向给定提示增补一组额外标记，使大语言模型基于这些标记获得修改后的分布；或（2）在输出层面：从标准语言模型中采样多个候选输出，并对这些候选输出进行处理。换言之，既可以修改由大语言模型自身诱导的提议分布，使其优于朴素地基于提示的条件分布；也可以使用事后验证器或评分器来执行输出修改。这一过程类似于马尔可夫链蒙特卡洛（MCMC）[2] 从复杂目标分布中采样，但结合了简单的提议分布与评分函数。通过改变输入标记直接修改提议分布，以及使用验证器，构成了本文研究的两个独立维度。

修改提议分布。改进提议分布的一种方法是，通过受强化学习启发的微调方法（如 STaR 或 ReST）直接针对给定的推理任务优化模型 π_{LM} [35, 50]。需要注意的是，这些技术并不利用任何额外的输入标记，而是专门微调

模型以诱导出改进的提议分布。相反，诸如自我批判 [4, 8, 23, 30] 之类的技术使模型能够在测试时通过指示其以迭代方式批判和修订自身输出来改进自身的提议分布。由于提示现成模型在测试时无法有效实现修订，我们专门微调模型，使其在复杂的基于推理的设置中迭代修订答案。为此，我们采用在策略数据上进行微调的方法，并利用最佳 N 项引导改进模型响应 [28]。

优化验证器。在我们对提议分布和验证器的抽象中，验证器用于从提议分布中聚合或选择最佳答案。使用此类验证器最典型的方式是应用最佳 N 项采样，即采样 N 个完整解，然后根据验证器选择最佳解 [7]。然而，通过训练基于过程的验证器 [22] 或过程奖励模型（PRM），可以进一步改进这种方法，该模型对解中每个中间步骤的正确性进行预测，而不仅仅是最终答案。然后，我们可以利用这些逐步预测在解空间上执行树搜索，与朴素的最佳 N 项相比，这可能实现更高效、更有效的针对验证器的搜索 [6, 10, 48]。

3. 如何最优地扩展测试时计算

鉴于各种方法的统一，我们现在希望了解如何最有效地利用测试时计算来提升语言模型在给定提示上的性能。具体而言，我们希望回答：

Problem setup

给定一个提示和一个测试时计算预算，在此预算内解决问题。在上述抽象下，存在不同的测试时计算利用方式。根据所给具体问题的不同，每种方法可能或多或少有效。我们如何确定针对给定提示最有效的测试时计算利用方式？与简单地使用更大的预训练模型相比，这种方法的表现如何？

在优化提议分布或针对验证器进行搜索时，有多种不同的超参数可以调整，以决定测试时计算预算的分配方式。例如，当使用经过微调以进行修订的模型作为提议分布，并使用结果奖励模型（ORM）作为验证器时，我们可以将全部测试时计算预算用于从模型并行生成 N 个独立样本，然后应用 N 选优；也可以使用修订模型按顺序采样 N 次修订，然后用 ORM 选择序列中的最佳答案；或者在这些极端之间取得平衡。直观上，我们可能预期“较容易”的问题从修订中获益更多，因为

模型的初始样本更有可能已经走在正确的轨道上，只是需要进一步细化。另一方面，具有挑战性的问题可能需要更多探索不同的高层次问题解决策略，因此在这种情况下，并行独立采样多次可能更为可取。对于验证器，我们还可以选择不同的搜索算法（例如束搜索、前瞻搜索、N 选优），每种算法根据手头验证器和提议分布的质量可能表现出不同的特性。与更简单的 N 选优或多数基线相比，更复杂的搜索过程在更难的问题上可能更有用。

3.1. 测试时计算最优扩展策略

因此，一般来说，我们希望针对给定问题选择测试时计算预算的最优分配。为此，对于任何给定的利用测试时计算的方法（例如本文中的修订和针对验证器的搜索，以及其他地方的各种方法），我们将“测试时计算最优扩展策略”定义为在测试时为给定提示选择与给定测试时策略相对应的超参数以获得最大性能收益的策略。形式上，定义目标分布 (θ, N, a) 为模型在给定提示 a 下、使用测试时计算超参数 θ 和计算预算 N 所诱导的自然语言输出标记的分布。我们希望选择超参数 θ ，以最大化给定问题的目标分布准确率。我们将其形式化表示为：

$$\theta_{q,a^*(q)}^*(N) = \operatorname{argmax}_{\theta} \left(\mathbb{E}_{y \sim \text{Target}(\theta, N, q)} [\mathbb{1}_{y=y^*(q)}] \right), \quad (1)$$

其中 $y^*(a)$ 表示 a 的真实正确响应， $\theta_{a^*(q)}^*(N)$ 表示在计算预算 N 下针对问题 q 的测试时计算最优扩展策略。

3.2. 为计算最优扩展估计问题难度

为了有效分析第 2 节中讨论的不同机制（例如提议分布和验证器）的测试时扩展特性，我们将规定该最优策略 θ^* 的一个近似，作为给定提示的某个统计量的函数。该统计量估计给定提示 $y^*(q)(N)$ 的难度概念。计算最优策略定义为该提示难度的函数。尽管这只是方程 1 所示问题的近似解，我们发现它仍能相对于以临时或均匀采样方式分配推理时计算的基线策略，带来显著的性能提升。

我们对问题难度的估计将给定问题分配到五个难度级别之一。然后，我们可以利用这种离散难度分类，在给定测试时计算预算下，在验证集上估计 θ^* 。随 $y^*(q)(N)$ 后，我们将这些计算最优策略应用于测试集。具体而言，我们为每个难度区间独立选择性能最佳的测试时计算策略。这样，在设计计算最优策略时，问题难度充当问题的充分统计量。

定义问题难度。遵循 Lightman 等人 [22] 的方法，我们将问题难度定义为给定基础大语言模型 (LLM) 的函数。具体来说，我们将模型在测试集中每个问题上的 pass@1 率（由 2048 个样本估计）划分为五个分位数，每个分位数对应递增的难度级别。我们发现，与 MATH 数据集中人工标注的难度区间相比，这种模型特定的难度区间概念更能预测使用测试时计算的有效性。尽管如此，我们注意到，如上所述评估问题难度假设可以访问真正正确性检查函数，而这在部署时当然不可用，因为此时我们

仅能访问我们不知道答案的测试提示。为了在实践中可行，一种以难度为条件的计算最优扩展策略需要首先评估难度，然后利用合适的扩展策略来解决该问题。因此，我们通过模型预测的难度概念来近似问题的难度，该概念对同一组每个问题 2048 个样本上学习到的验证器（而非真实答案正确性检查）的平均最终答案分数执行相同的分箱过程。我们将此设置称为模型预测难度，将依赖真实正确性的设置称为预言机难度。虽然模型预测难度消除了了解真实标签的需要，但以这种方式估计难度在推理过程中仍会产生额外的计算成本。尽管如此，这种一次性的推理成本可以包含在实际运行推理时策略的成本中（例如，当使用验证器时，可以使用相同的推理计算来运行搜索）。更一般地说，这类似于强化学习中的探索-利用权衡：在实际部署条件下，我们必须平衡用于评估难度的计算与应用计算最优方法的计算。这是未来工作的关键方向（见第 8 节），我们的实验在很大程度上为了简单起见没有考虑这一成本，因为我们的目标是展示通过有效分配测试时计算实际上可能实现的一些初步结果。为了避免使用相同的测试集来计算难度分箱和选择计算最优策略所带来的混淆，我们在测试集中的每个难度分箱上使用两折交叉验证。我们根据一折上的性能选择最佳策略，然后在另一折上使用该策略测量性能，反之亦然，对两个测试折的结果取平均。

4. 实验设置

我们首先概述进行此分析的实验设置，包括多种验证器设计选择和提议分布，随后在后续章节中给出分析结果。

数据集。我们预期，当模型已经具备回答问题所需的所有基本“知识”，而主要挑战在于从这些知识中进行（复杂）推理时，测试时计算将最为有用。为此，我们聚焦于 MATH [13] 基准，该基准包含难度各异的高中竞赛级数学问题。在所有实验中，我们使用 Lightman 等人 [22] 所使用的数据集划分，包含 12k 训练样本和 500 测试问题。

模型。我们使用 PaLM 2-S* [3] (Codey) 基础模型进行分析。我们认为该模型能够代表许多当代大语言模型的能力，因此我们的发现很可能适用于类似模型。最重要的是，该模型在 MATH 上取得了不俗的性能且尚未饱和，因此我们预期该模型将为我们提供一个良好的测试平台。

5. 通过验证器扩展测试时计算

在本节中，我们分析如何通过尽可能有效地优化验证器来扩展测试时计算。为此，我们研究使用过程验证器 (PRM) 执行测试时搜索的不同方法，并分析这些不同方法的测试时计算扩展特性。

5.1. 训练适用于搜索的验证器

PRM 训练。最初的 PRM 训练 [22, 42] 使用人工众包标注。虽然 Lightman 等人 [22] 发布了他们的 PRM 训练数据（即 PRM800k 数据集），但我们发现该数据在很大程度上

对我们无效。我们发现，即使采用朴素策略（如 N 选优采样），也容易利用在该数据集上训练的过程奖励模型。我们推测，这可能是由于他们数据集中 GPT-4 生成的样本与我们的 PaLM 2 模型之间存在分布偏移。与我们的 PaLM 2 模型收集众包工作者的过程奖励模型标签这一昂贵过程，我们转而采用 Wang 等人 [45] 的方法，在无需人工标签的情况下监督过程奖励模型，利用从解答中每一步运行蒙特卡洛展开得到的逐步正确性估计。因此，我们的过程奖励模型的逐步预测对应于基础模型采样策略的奖励剩余价值估计，与近期工作 [31, 45] 类似。我们还与结果奖励模型基线（附录 F）进行了比较，但发现我们的过程奖励模型始终优于结果奖励模型。因此，本节所有搜索实验均使用过程奖励模型。过程奖励模型训练的更多细节见附录 D。

答案聚合。在测试时，基于过程的验证器可用于对从基础模型采样的一组解答中的每个单独步骤进行评分。为了利用过程奖励模型选择 N 选优答案，我们需要一个函数来聚合每个答案的所有逐步得分，以确定正确答案的最佳候选。为此，我们首先聚合每个单独答案的逐步得分，以获得完整答案的最终得分（步骤级聚合）。然后，我们在答案之间进行聚合，以确定最佳答案（答案间聚合）。具体而言，我们按如下方式处理步骤级和答案间聚合：

- **步骤级聚合。**我们不是通过取乘积或最小值 [22, 45] 来聚合逐步得分，而是使用过程奖励模型在最后一部的预测作为完整答案得分。我们发现，在我们研究的所有聚合方法中，这种方法表现最佳（见附录 E）。
- **答案间聚合。**我们遵循 Li 等人 [21] 的方法，应用“ N 选优加权”选择，而非标准 N 选优。 N 选优加权选择将验证器对所有具有相同最终答案的解答的正确性得分进行边缘化，选择总得分最高的最终答案。

5.2. 针对过程奖励模型的搜索方法

本文在测试阶段通过搜索方法优化过程奖励模型（PRM）。我们研究了三种搜索方法，这些方法从少样本提示的基础大语言模型（LLM）中采样输出（见附录 G）。图 2 给出了示意图。

加权最佳 N 选（Best-of- N weighted）。我们从基础大语言模型中独立采样 N 个答案，然后根据过程奖励模型对最终答案的判断选择最佳答案。

束搜索（Beam Search）。束搜索通过搜索过程奖励模型的逐步预测来优化它。我们的实现类似于 BFS-V [10, 48]。具体来说，我们考虑固定数量的束 N 和束宽。然后执行以下步骤：
 M

1. 为解的第一步采样 N 个初始预测；
2. 根据过程奖励模型预测的逐步奖励到终局估计（由于该设置下奖励稀疏，这也对应于前缀的总奖励）对生成的步骤进行评分；
3. 仅保留评分最高的前 N/M 个步骤；
4. 现在从每个候选中采样下一步的 M 个提议，再次得到总共 $N/M \times M$ 个候选前缀。然后重复步骤 2-4。我们运行该算法直到解结束或达到束扩展的最大轮数（本文为 40 轮）。搜索结束时得到 N 个最终答案候选，然后应用上述加权最佳 N 选方法进行最终答案预测。

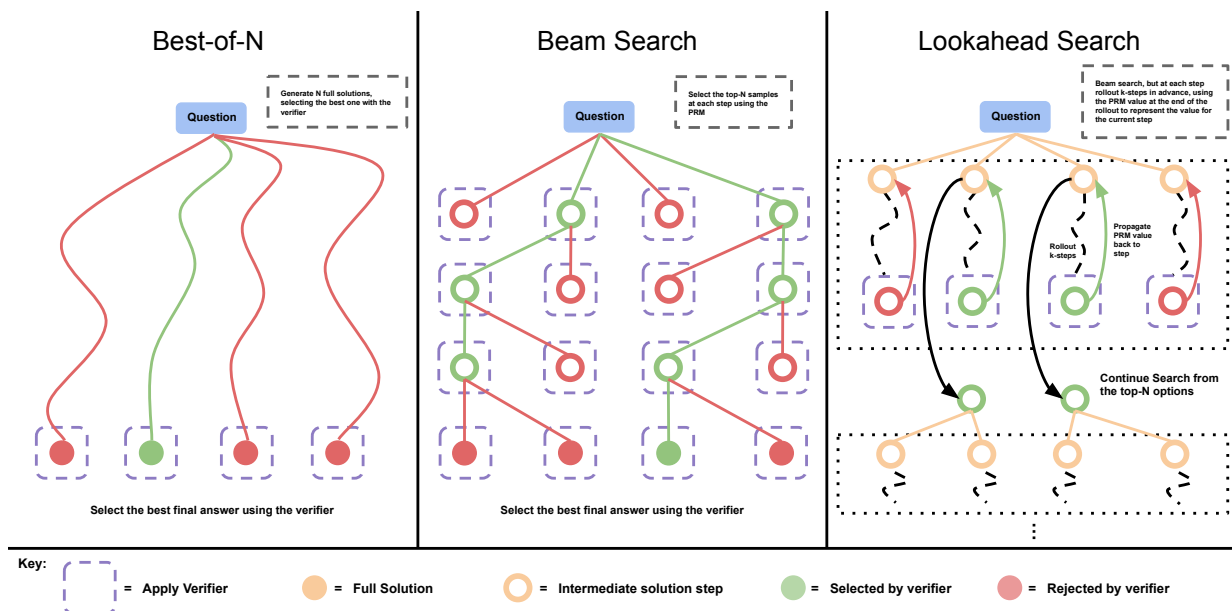


图 2 | 比较不同的过程奖励模型搜索方法。左：最佳 N 选采样 N 个完整答案，然后根据过程奖励模型最终得分选择最佳答案。中：束搜索在每一步采样 N 个候选，并根据过程奖励模型选择前 M 个继续搜索。右：前瞻搜索扩展束搜索中的每一步，在评估保留哪些步骤并继续搜索时利用 k 步前瞻。因此前瞻搜索需要更多计算。

前瞻搜索 (Lookahead Search)。前瞻搜索改变了束搜索评估单个步骤的方式。它利用前瞻推演来提高搜索过程中每一步的 PRM 价值估计的准确率。具体而言，在束搜索的每一步中，前瞻搜索不是使用当前步骤的 PRM 分数来选择最佳候选，而是执行一次仿真，最多向前推演 k 步，如果到达解答末尾则提前停止。为了最小化仿真推演中的方差，我们使用温度 0 进行推演。然后，使用该推演结束时 PRM 的预测来对束搜索中的当前步骤进行评分。换句话说，我们可以将束搜索视为前瞻搜索在 $k = 0$ 时的特例。在 PRM 准确的情况下，增加 k 应该会提高每步价值估计的准确率，但代价是额外的计算量。还需注意，这种前瞻搜索版本是蒙特卡洛树搜索 (MCTS) [38] 的一个特例，其中为了促进探索而设计的 MCTS 随机元素被移除，因为 PRM 已经训练好且被冻结。这些随机元素主要用于学习价值函数（我们已经通过 PRM 学到了），但在测试时，当我们想要利用而非探索时，它们的作用较小。因此，前瞻搜索在很大程度上代表了 MCTS 类方法在测试时的应用方式。

5.3. 分析结果：带验证器的搜索的测试时扩展

我们现在展示比较各种搜索算法的结果，并确定一种依赖于提示难度的计算最优扩展策略，用于搜索方法。

比较搜索算法。 本文首先对各种搜索设置进行扫描。除了标准的最优 N 选一方法外，我们还扫描了区分不同树搜索方法的两个主要参数：束宽 M 和前瞻步数 k 。虽然无法对每一种配置进行详尽扫描，但在最大预算为 256 的条件下，我们扫描了以下设置：1) 束搜索，束宽设为 N 为生成预算。2) 束搜索，固定束宽为 4。

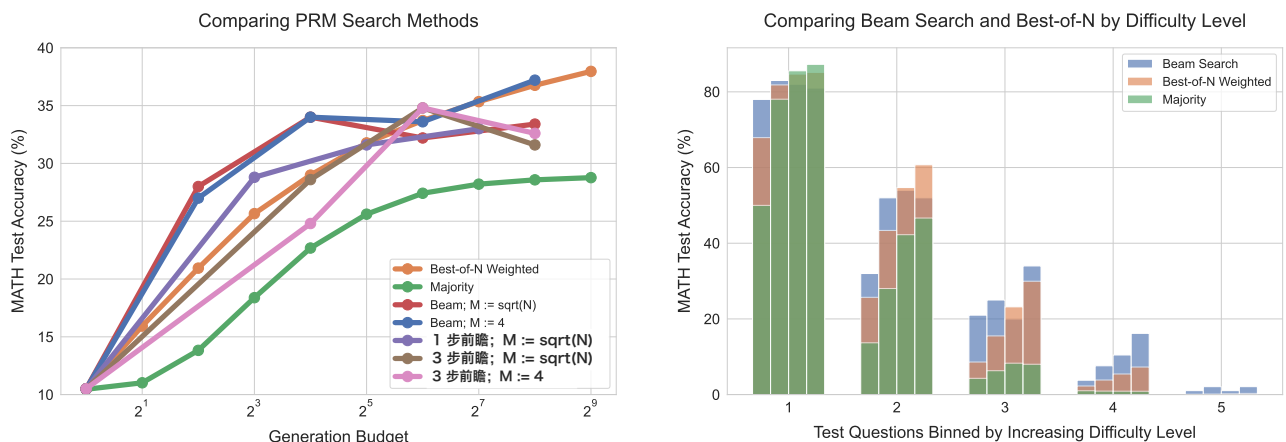


图 3 | 左：比较针对过程奖励模型验证器进行搜索的不同方法。我们观察到，在低生成预算下，束搜索表现最佳，但随着预算进一步扩大，其改进幅度逐渐减小，最终低于 N 中择优基线。前瞻搜索在相同生成预算下通常表现不如其他方法。右：按难度级别分箱比较束搜索与 N 中择优。每个难度箱中的四个柱状条对应递增的测试时计算预算（4、16、64 和 256 次生成）。在较简单的问题（箱 1 和箱 2）上，束搜索在较高预算下显示出过度优化的迹象，而 N 中择优则没有。在中等难度的问题（箱 3 和箱 4）上，我们看到束搜索相对于 N 中择优表现出持续的改进。

3) 将 $k = 3$ 的前瞻搜索应用于束搜索设置 1) 和 2)。

4) 将 $k = 1$ 的前瞻搜索应用于束搜索设置 1)。

为了公平地比较不同搜索方法随生成预算变化的表现，我们构建了一个估算每种方法成本的协议。我们将一次生成视为从基础大语言模型中采样得到的一个答案。对于束搜索和 N 中择优，生成预算分别对应于束的数量和 N 。然而，前瞻搜索会利用额外的计算：在搜索的每一步，我们会额外采样 k 步前瞻。因此，我们将前瞻搜索的成本定义为 $N \times (k + 1)$ 个样本。

结果。如图 3（左）所示，在较小的生成预算下，束搜索显著优于 N 中择优。然而，随着预算扩大，这些改进大幅减弱，束搜索常常表现不如 N 中择优基线。我们还观察到，前瞻搜索在相同生成预算下通常表现不如其他方法，这可能是由于模拟前瞻展开所引入的额外计算所致。搜索带来的收益递减可能是由于对过程奖励模型预测的利用。例如，我们观察到一些实例（如图 29 所示），其中搜索导致模型在解答末尾生成低信息量的重复步骤。在其他情况下，我们发现过度优化搜索可能导致解答过短，仅包含 1-2 个步骤。这解释了为何最强大的搜索方法（即前瞻搜索）表现最差。我们在附录 M 中收录了搜索发现的若干此类示例。

搜索改进了哪些问题？ 为了理解如何以计算最优的方式扩展搜索方法，本文进行了难度分箱分析。具体而言，本文将束搜索（ $M = 4$ ）与最优 N 选一（best-of- N ）进行对比。在图 3（右）中可以看到，尽管在总体上，束搜索与最优 N 选一在高生成预算下表现相似，但按难度分箱评估其效果时，却呈现出截然不同的趋势。在简单问题（级别 1 和 2）上，两种方法中优化能力更强的束搜索随着生成预算的增加反而导致性能下降，这表明其可能利用了过程奖励模型（PRM）信号中的虚假特征。相比之下，在较难问题（级别 3 和 4）上，束搜索始终优于最优 N 选一。在最难问题（级别 5）上，两种方法均未取得显著进展。

这些发现与直觉相符：在简单问题上，验证器对正确性的判断大多是正确的。因此，通过束搜索进行进一步优化，只会进一步放大验证器学习到的虚假特征，从而导致性能下降。而在较难问题上，基础模型本身采样到正确答案的概率较低，因此搜索可以起到引导作用，帮助模型更频繁地生成正确答案。

计算最优搜索。鉴于上述结果，问题难度显然可以作为预测在给定计算预算下最优搜索策略的有用统计量。此外，最优搜索策略的选择会随该难度统计量的变化而发生显著变化。因此，本文在图 4 中可视化了“计算最优”扩展趋势，即每个难度级别上表现最佳的搜索策略。可以看到，在低生成预算区间，无论是使用真实难度还是预测难度，计算最优扩展几乎都能以最多 4 倍少的测试时计算量（例如 16 次生成对比 64 次生成）超越最优 N 选一。而在较高预算区间，使用预测难度时部分优势有所减弱，但使用真实难度分箱时，最优扩展测试时计算仍能带来持续改进。这一结果表明，在搜索过程中自适应地分配测试时计算量可以带来性能提升。

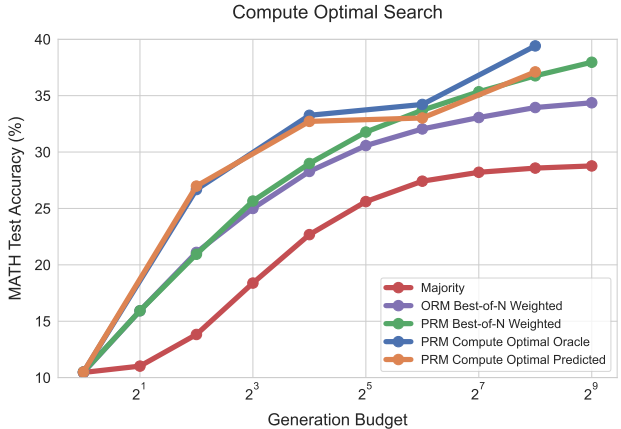


Figure 4 | Comparing compute-optimal test-time compute allocation against baselines with PRM search. By scaling test time compute per the notion of question difficulty, we find that we can nearly outperform PRM best-of-N using up to 4× less test-time compute (e.g. 16 verses 64 generations). “**Compute-optimal oracle**” refers to using oracle difficulty bins derived from the groundtruth correctness information, and “**compute-optimal predicted**” refers to using the PRM’s predictions to generate difficulty bins. Observe that the curves with either type of difficulty bins largely overlap with each other.

Takeaways for compute-optimal scaling of verifiers

我们发现，任何给定验证器搜索方法的有效性都关键取决于计算预算和所处理的问题。具体而言，束搜索在较难的问题和较低的计算预算下更为有效，而 N 选优在较易的问题和较高的预算下更为有效。此外，通过针对给定的问题难度和测试时计算预算选择最佳搜索设置，我们几乎可以用少至 4 倍的测试时计算量超越 N 选优。

6. 优化提议分布

到目前为止，我们研究了针对验证器进行搜索时的测试时计算扩展特性。现在，我们转而研究修改提议分布（第 2 节）的扩展特性。具体来说，我们使模型能够迭代地修正自己的答案，从而在测试时动态改进其自身的分布。简单地提示现有的大语言模型纠正自己的错误，在推理问题上往往对提升性能收效甚微 [15]。因此，我们基于 Qu 等人 [28] 提出的方法，结合我们的设置进行修改，并对语言模型进行微调，使其能够迭代地修正自己的答案。我们首先描述如何训练和使用那些通过顺序地以自己先前对问题的尝试为条件来优化自身提议分布的模型。然后，我们分析修正模型的推理时扩展特性。

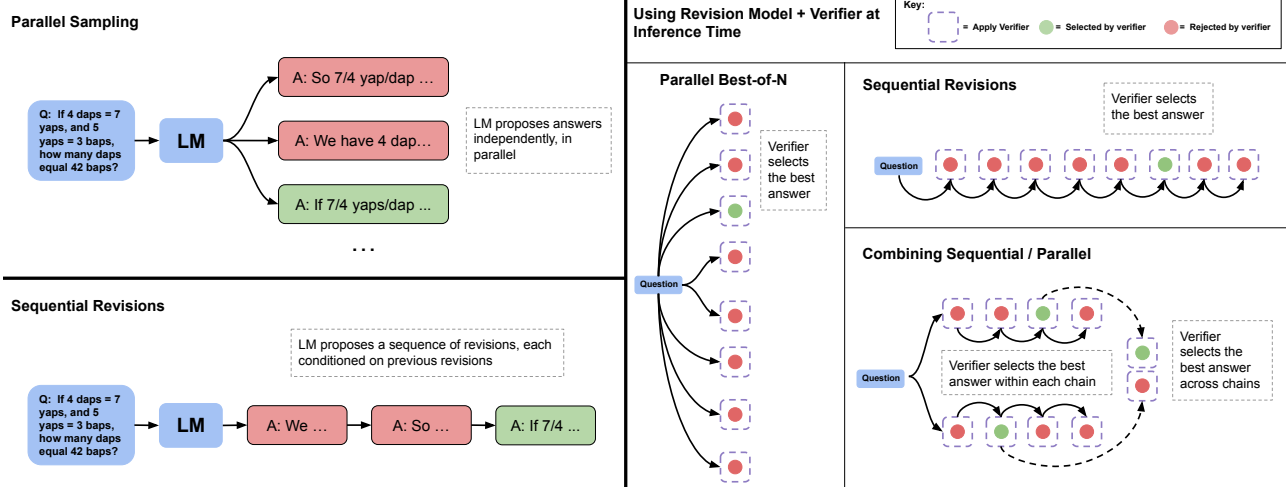


图 5 | 并行采样（例如，N 选优）与顺序修正的对比。左图：并行采样独立地并行生成 N 个答案，而顺序修正则依次生成每个答案，并以先前的尝试为条件。右图：在顺序和并行两种情况下，我们都可以使用验证器来确定 N 选优答案（例如，通过应用加权 N 选优）。我们还可以将部分预算分配给并行采样，部分分配给顺序修正，从而有效地实现两种采样策略的结合。在这种情况下，我们首先使用验证器在每个顺序链中选择最佳答案，然后在各链之间选择最佳答案。

6.1. 设置：训练和使用修正模型

我们的修订模型微调流程与 [28] 类似，但引入了一些关键差异。微调需要由一系列错误答案后跟一个正确答案组成的轨迹，然后可以对其进行监督微调（SFT）。理想情况下，我们希望正确答案与上下文中提供的错误答案相关，从而有效地教会模型隐式识别上下文示例中的错误，然后通过编辑来纠正这些错误，而不是完全忽略上下文示例并从头开始重试。

生成修订数据。 Qu 等人 [28] 提出的用于获取多个多轮展开的在线策略方法被证明是有效的，但由于运行多轮展开相关的计算成本，在我们的基础设施中并不完全可行。因此，我们在较高温度下并行采样 64 个响应，并从这些独立样本中事后构建多轮展开。具体来说，按照 [1] 的方法，我们将每个正确答案与来自该集合的一系列错误答案配对作为上下文，以构建多轮微调数据。我们在上下文中最多包含四个错误答案，其中上下文中解决方案的具体数量从 0 到 4 的均匀分布中随机采样。我们使用字符编辑距离度量来优先选择与最终正确答案相关的错误答案（见附录 H）。请注意，标记编辑距离并不是相关性的完美度量，但我们发现这种启发式方法足以将上下文中的错误答案与正确的目标答案相关联，从而促进训练有意义的修订模型，而不是将不相关的错误和正确响应随机配对。

在推理时使用修订。 给定一个微调后的修订模型，我们可以在测试时从模型中采样一系列修订。虽然我们的修订模型仅在上下文中最多包含四个先前答案的情况下进行训练，但我们可以通过将上下文截断为最近四个修订响应来采样更长的链。在图 6（左）中，我们看到随着从修订模型中采样更长的链，模型每一步的 pass@1 逐渐提高，表明我们能够有效地教会模型从上下文中先前答案所犯的错误中学习。

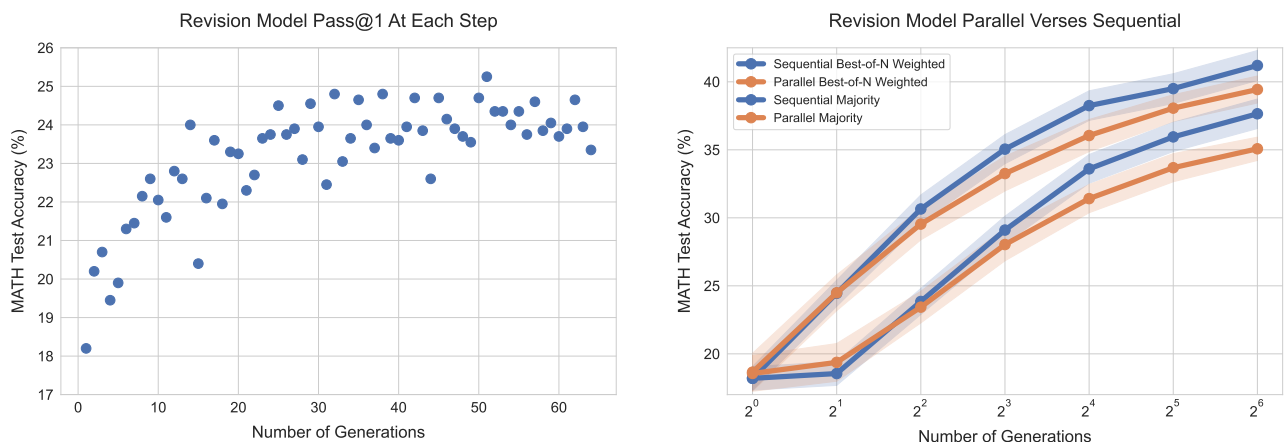


图 6 | 左：本文修订模型在每个修订步骤的通过率@1。通过率@1 在每个修订步骤后逐步提升，甚至超过了其训练时的 4 个修订步骤。本文通过平均测试集中每个问题的 4 条长度为 64 的修订轨迹的性能来估计每个步骤的通过率@1。右：顺序采样与并行采样

来自修订模型。比较使用本文修订模型并行生成 N 个初始答案与使用该模型顺序生成 N 个修订版本的性能。当同时使用验证器和多数投票来选择答案时，本文发现使用修订模型顺序生成答案略优于并行生成。

然而，在推理时存在分布偏移：模型仅在上下文中包含错误答案的序列上进行训练，但在测试时模型可能会采样到正确答案并将其包含在上下文中。在这种情况下，它可能会在下一个修订步骤中意外地将正确答案改为错误答案。本文发现，与 Qu 等人 [28] 类似，使用朴素方法时，本文修订模型将约 38% 的正确答案改回错误答案。因此，本文采用基于顺序多数投票或基于验证器选择的机制，从模型生成的修订序列中选择最正确的答案（见图 5），以产生最佳答案。

比较。为了测试通过修订修改提议分布的有效性，本文在顺序采样 N 个修订版本与并行采样 N 次尝试的性能之间设置了公平比较。在图 6（右）中可以看到，使用基于验证器和基于多数的选择机制时，顺序采样解决方案优于并行采样。

6.2. 分析结果：通过修订实现测试时扩展

我们之前看到，顺序提出答案优于并行提出答案。然而，我们可能预期顺序采样和并行采样具有不同的特性。并行采样答案可能更像一种全局搜索过程，原则上可以覆盖许多完全不同的解决问题的方法，例如，不同的候选答案可能采用完全不同的高层方法。另一方面，顺序采样可能更像一种局部细化过程，修订已经在一定程度上走上正轨的响应。由于这些互补的优势，我们应该在这两个极端之间取得平衡，将部分推理时间预算分配给并行采样（例如 $\frac{b\epsilon}{\sqrt{N}}$ ），其余分配给顺序修订（例如 $\frac{1}{\sqrt{N}}$ ）。我们现在将展示顺序采样和并行采样之间存在计算最优比例，并根据给定提示的难度理解它们的相对优缺点。

权衡顺序和并行测试时计算。为了理解如何最优分配顺序和并行计算，我们对多种不同比例进行了扫描。我们在图 7（左）中看到，确实，在给定的生成预算下，存在一个理想的顺序与并行比例，

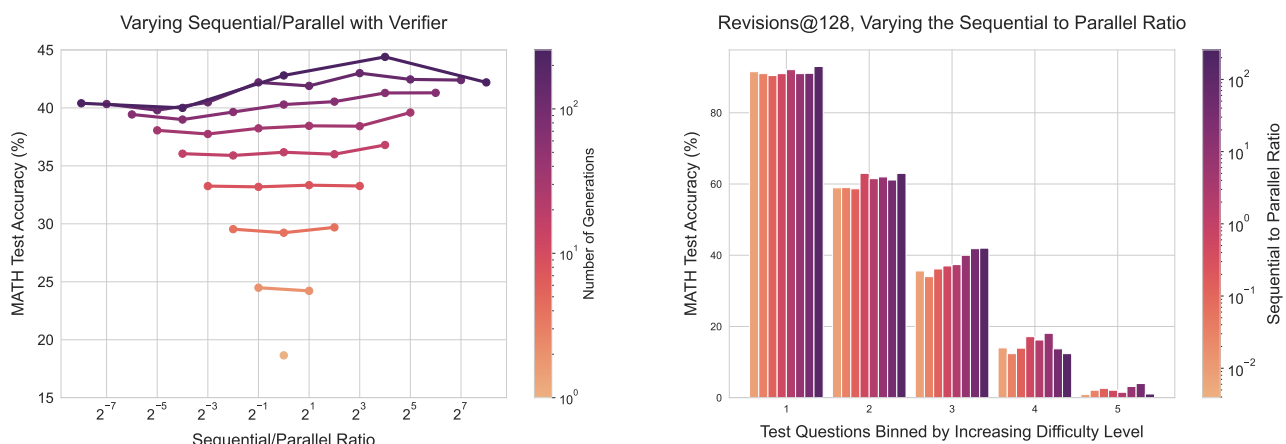


图 7 | 左：改变分配给顺序修订与并行样本的生成预算比例。每条线代表一个固定的生成预算，随着比例的变化而变化。我们使用验证器进行答案选择。我们看到，虽然增加顺序修订往往优于更多的并行计算，但在更高的生成预算下，存在一个理想的比例，在两个极端之间取得平衡。右：在生成预算为

128 时，跨难度区间改变顺序与并行比例。使用基于验证器的选择，我们看到较容易的问题在完全顺序计算下获得最佳性能。在较难的问题上，存在一个理想的顺序与并行测试时计算比例。

达到最大准确率。从图 7（右）中还可以看出，顺序与并行计算的理想比例随问题难度而变化。具体而言，简单问题从顺序修订中获益更多，而困难问题则需要在顺序与并行计算之间取得平衡。这一发现支持了以下假设：顺序修订（即改变提议分布）与并行采样（即使用验证器进行搜索）是扩展测试时计算规模的两个互补维度，且按提示进行针对性调整可能更为有效。模型生成示例见附录 L，更多结果见附录 B。

计算最优修订。鉴于顺序与并行采样的效果取决于问题难度，我们可以为每个难度分箱选择顺序与并行计算的理想比例。在图 8 中，我们展示了在采用预言和预测两种难度概念时，使用该计算最优扩展策略的结果。在两种情况下，我们都能够通过修订改进提议分布，从而显著改善测试时计算扩展。特别是，在较高的生成预算下，并行采样似乎趋于平台期，而计算最优扩展则持续改进。对于预言和预测难度分箱，计算最优扩展均能以最多减少 4 倍的测试时计算量（例如 64 个样本对比 256 个样本）超越最优 N 采样。总体而言，这些结果表明，通过按提示调整提议分布，测试时计算扩展具有改进潜力。

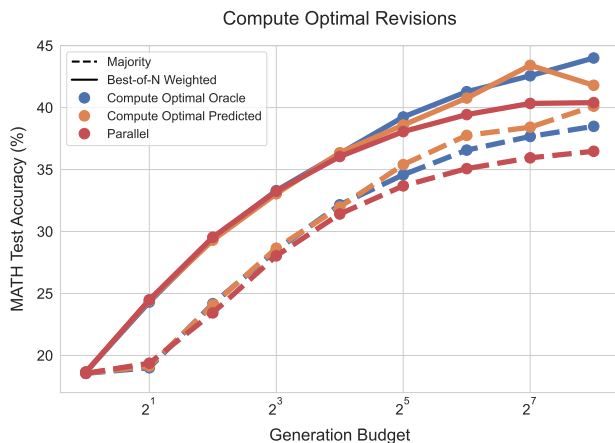


图 8 | 将计算最优的测试时计算分配与使用修订模型的并行计算基线进行比较。通过根据问题难度最优地扩展测试时计算，我们发现可以用最多减少 4 倍的测试时计算量（例如 64 个样本对比 256 个样本）超越最优 N 采样。“计算最优预言”指使用由真正正确性信息得出的预言难度分箱，“计算最优预测”指使用过程奖励模型的预测来生成模型预测的难度分箱。

Takeaways for compute-optimal scaling by refining the proposal distribution with revisions

我们发现，在顺序（例如修订）与并行（例如标准最优 N）测试时计算之间存在权衡，顺序与并行测试时计算的理想比例关键取决于计算预算和具体问题。具体而言，较简单的问题受益于纯顺序测试时计算，而较难的问题通常在顺序与并行计算的某种理想比例下表现最佳。此外，通过针对给定问题难度和测试时计算预算最优选择设置，我们可以使用最多减少 4 倍的测试时计算超越并行最优 N 基线。

7. 综合：交换预训练与测试时计算

到目前为止，我们看到利用额外的测试时计算可以表示比基础大语言模型本身预测的更复杂的分布，从而提高性能。我们现在假设，这种表示分布的增强灵活性意味着我们可以期望额外的测试时计算弥补预训练期间缺乏更高容量模型或更多浮点运算训练。在本节中，我们研究这种可能性有多大。我们提出以下问题：

Question: Exchanging pretraining and test-time compute

假设一个模型使用 X 浮点运算进行预训练。假设我们计划使用该模型运行 Y 浮点运算的推理。如果我们想通过将总浮点运算预算增加 M (i.e., $M(X+Y)$) 倍（跨预训练和推理的总浮点运算）来提高性能，我们应该将浮点运算用于增加预训练计算还是额外的测试时计算？

增加预训练浮点运算引入了额外的设计决策：是将计算分配给更多数据还是更多参数 [14]。我们专注于模型参数扩大而训练数据量固定的设置，这与开源 LLaMA 系列模型采用的方法一致 [41]。我们选择此设置是因为它代表了扩展预训练计算的典型方法，并将数据和参数均等扩展的计算最优扩展预训练计算 [29] 的分析留给未来工作。

定义浮点运算之间的交换率。我们现在描述如何定义预训练与推理浮点运算次数之间的换算关系。为确定预训练浮点运算次数，采用常用近似 $X = 6ND_{\text{pretrain}}$ [14]，而对于推理浮点运算次数，采用 $Y = 2ND_{\text{inference}}$ [29]。此处 N 表示模型参数量， D_{pretrain} 为预训练所用词元数， $D_{\text{inference}}$ 为推理时生成的总词元数。借助这些近似，可以看出，若将模型参数量乘以因子 M ，则预训练与推理浮点运算次数（由于使用更大模型进行贪心解码的成本）均增加因子 M （总计 $M(X+Y)$ 浮点运算次数）。为使较小模型通过测试时计算匹配扩大模型参数量带来的浮点运算次数，可将较小模型的推理计算乘以因子 $M + 3\left(\frac{\nu_{\text{pretrain}}}{\nu_{\text{inference}}}\right)(M-1)$ 。值得注意的是，可用于匹配较大模型浮点运算次数的推理计算量取决于比值 ν_{pretrain} 本文将这一比率的逆矩阵称为 R (e.g.

$\frac{1}{\nu_{\text{pretrain}}}$)。根据具体的生产环境或使用场景，我们应当预期 R 的值会有很大差异。特别是在许多大规模生产环境中，我们可能预期推理令牌数量显著多于预训练令牌，此时 $R \gg 1$ 。另一方面，在许多当代自我改进设置中，利用测试时计算来改进模型，我们生成的令牌数量可能显著减少

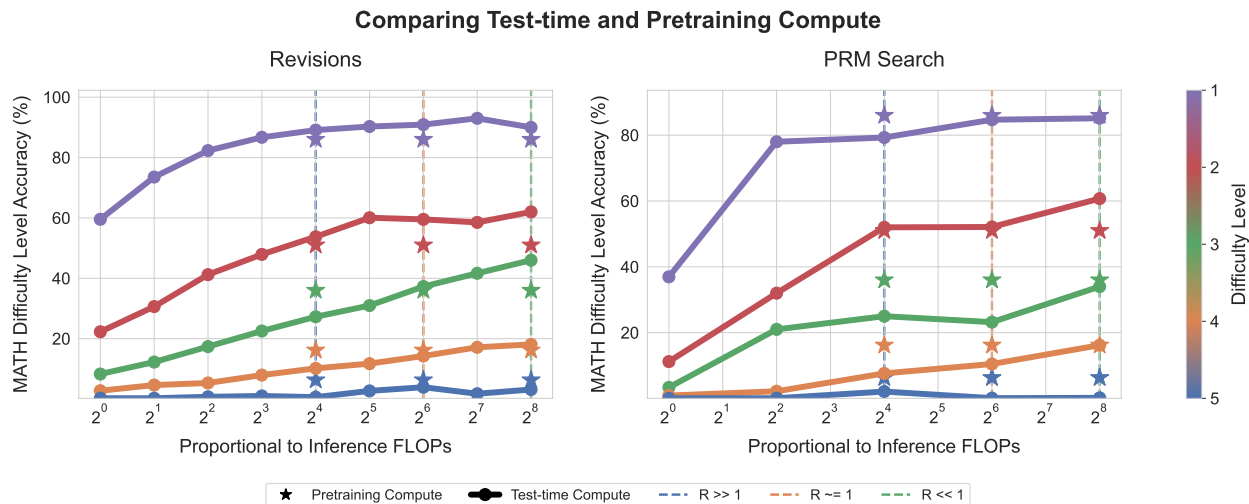


图9 在浮点运算次数匹配的评估中，预训练与测试时计算之间的权衡。每条线代表在每个难度区间内，采用本文计算最优策略扩展测试时计算的性能。左侧为修订结果，右侧为搜索结果。星号表示预训练参数量多 ~ 14 倍的基础模型的贪心一次通过性能。横轴为测试时计算预算，星号置于横轴三个不同位置，分别对应三种不同推理计算负载下扩展参数与扩展测试时计算之间的浮点运算次数等效比较点（例如 $R = \frac{D_{inference}}{D_{pretraining}}$ ）。若星号位于线下方，则表明使用测试时计算比扩展模型参数更有效；若星号位于线上方，则表明扩展参数更有效。可见，在简单问题或推理负载较低的设置中（e.g. $R \ll 1$ ），测试时计算通常能优于扩展模型参数。然而，在较难问题或推理负载较高的设置中（例如 $R \gg 1$ ），预训练是提升性能的更有效方式。

推理令牌多于预训练令牌，得到 $R \ll 1$ 。因此，由于可应用的测试时计算规模取决于该比率，我们预期具体设置不同会得出不同结论。

在图9中，我们采用这种交换测试时与预训练计算的方法，将本文计算最优扩展与参数量扩大 ~ 14 倍的模型进行比较。我们针对三个不同的 R 值进行比较：0.16 ($R \ll 1$)、0.79 ($R \sim 1$) 和 22 ($R \gg 1$)，每个比率对应一个推理预算。观察到，若预期只会遇到非常困难的问题（例如难度区间 4/5）或具有较大的 $D_{inference}$ （对应较大的 R 值），则通常将预算分配给预训练更有效（例如星号位于线上方）。反之，若预期主要是简单或中等难度的问题（例如区间 1/2/3，有时包括 4）或推理要求较低（如自改进流程中的情况），则利用测试时计算更佳。

Takeaways for exchanging pretraining and test-time compute

测试时计算与预训练计算并非一一对应地“可互换”。在模型能力范围内的简单和中等难度问题上，或在推理需求较小的场景中，测试时计算可以轻易弥补额外预训练的不足。然而，在超出给定基础模型能力的挑战性问题上，或在推理需求较高的场景下，预训练可能对提升性能更为有效。

8. 讨论与未来工作

在本工作中，我们对旨在改进针对验证器的搜索或优化大语言模型提议分布的不同技术的有效性进行了全面分析，以扩展测试时计算

用于数学推理。总体而言，我们发现给定方法的有效性与从基础大语言模型能力角度看问题的难度高度相关。这促使我们引入“计算最优”的测试时计算扩展概念，该概念规定了一种自适应的、依赖提示的策略，以在给定测试时计算预算下提升性能。通过应用这种计算最优扩展策略，我们发现可以将测试时计算扩展的效率提升 $2 - 4\times$ 倍。在浮点运算数匹配的设置下，将额外测试时计算带来的收益与额外预训练计算带来的收益进行比较时，我们首次表明，使用看似简单的方法（即修订和搜索）进行测试时计算，在某些类型的提示上已经能够很好地扩展，其收益超过了将这些浮点运算数用于预训练。尽管如此，我们的研究也存在一些局限性，未来工作可以致力于解决这些问题。

进一步改进测试时计算扩展。在本工作中，我们专注于改进两种主要机制的测试时计算扩展：验证器和提议分布（通过修订）。虽然我们在第 6 节中将验证器与修订相结合，但我们没有将过程奖励模型树搜索技术与修订相结合进行实验。我们也没有研究其他技术，如批判与修订 [23]。未来工作应研究如何通过结合多种此类方法来进一步改进测试时计算扩展。此外，我们发现这些方案在困难问题上普遍提供的增益较小；未来工作应致力于开发新的测试时计算使用方式，以规避这一局限性。

快速评估问题难度。我们采用了一种问题难度的概念，作为近似计算最优测试时扩展策略的简单充分统计量。虽然该方案有效，但估计我们所定义的难度本身就需要消耗相当数量的测试时计算资源。未来工作应考虑更高效地估计问题难度的替代方法（例如，通过预训练或微调模型直接预测问题难度），或在评估难度与尝试解题之间动态切换。

测试时与训练时计算的交织。本文仅聚焦于测试时计算扩展，以及测试时计算可在多大程度上替代额外的预训练。然而，未来我们设想，应用额外测试时计算所产生的输出可以被蒸馏回基础大语言模型，从而形成一个在开放式自然语言上运行的迭代自我改进循环。为此，未来工作应扩展我们的发现，研究如何利用测试时计算的输出来改进基础大语言模型本身。

致谢

我们感谢 Yi Su、Rishabh Agarwal、Yinlam Chow、Aleksandra Faust、Vincent Zhuang、George Tucker、Hao Liu、Jiayi Pan、Ethan Dyer、Behnam Neyshabur、Xavier Garcia、Yamini Bansal、Lampros Lamprou、Yuxiao Qu 和 Amrith Setlur 对本文早期版本提出的反馈与讨论。我们特别感谢 Rishabh Agarwal、Vincent Zhuang、Yi Su 和 Avi Singh 提供的想法、讨论以及实验，这些实验具体展示了成对样本生成在训练修订模型方面的前景，以及基于编辑距离的采样方法在 [1] 中的应用。我们感谢 Slav Petrov 的领导支持。

参考文献

- [1] Training revision models with synthetic data. Coming soon, 2024.

-
- [2] C. Andrieu, N. De Freitas, A. Doucet, and M. I. Jordan. An introduction to mcmc for machine learning. 2003.
- [3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, E. Chu, J. H. Clark, L. E. Shafey, Y. Huang, K. Meier-Hellstern, G. Mishra, E. Moreira, M. Omernick, K. Robinson, S. Ruder, Y. Tay, K. Xiao, Y. Xu, Y. Zhang, G. H. Abrego, J. Ahn, J. Austin, P. Barham, J. Botha, J. Bradbury, S. Brahma, K. Brooks, M. Catasta, Y. Cheng, C. Cherry, C. A. Choquette-Choo, A. Chowdhery, C. Crepy, S. Dave, M. Dehghani, S. Dev, J. Devlin, M. Díaz, N. Du, E. Dyer, V. Feinberg, F. Feng, V. Fienber, M. Freitag, X. Garcia, S. Gehrmann, L. Gonzalez, G. Gur-Ari, S. Hand, H. Hashemi, L. Hou, J. Howland, A. Hu, J. Hui, J. Hurwitz, M. Isard, A. Ittycheriah, M. Jagielski, W. Jia, K. Kenealy, M. Krikun, S. Kudugunta, C. Lan, K. Lee, B. Lee, E. Li, M. Li, W. Li, Y. Li, J. Li, H. Lim, H. Lin, Z. Liu, F. Liu, M. Maggioni, A. Mahendru, J. Maynez, V. Misra, M. Moussalem, Z. Nado, J. Nham, E. Ni, A. Nystrom, A. Parrish, M. Pellat, M. Polacek, A. Polozov, R. Pope, S. Qiao, E. Reif, B. Richter, P. Riley, A. C. Ros, A. Roy, B. Saeta, R. Samuel, R. Shelby, A. Slone, D. Smilkov, D. R. So, D. Sohn, S. Tokumine, D. Valter, V. Vasudevan, K. Vodrahalli, X. Wang, P. Wang, Z. Wang, T. Wang, J. Wieting, Y. Wu, K. Xu, Y. Xu, L. Xue, P. Yin, J. Yu, Q. Zhang, S. Zheng, C. Zheng, W. Zhou, D. Zhou, S. Petrov, and Y. Wu. Palm 2 technical report, 2023.
- [4] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan. Constitutional ai: Harmlessness from ai feedback, 2022.
- [5] C. Blakeney, M. Paul, B. W. Larsen, S. Owen, and J. Frankle. Does your data spark joy? performance gains from domain upsampling at the end of training, 2024. URL <https://arxiv.org/abs/2406.03476>.
- [6] G. Chen, M. Liao, C. Li, and K. Fan. Alphamath almost zero: process supervision without process, 2024.
- [7] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems, 2021.
- [8] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023.
- [9] J. S. B. T. Evans. Heuristic and analytic processes in reasoning. *British Journal of Psychology*, 75(4): 451–468, 1984.
- [10] X. Feng, Z. Wan, M. Wen, S. M. McAleer, Y. Wen, W. Zhang, and J. Wang. Alphazero-like tree-search can guide large language model decoding and training, 2024.
- [11] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. Pal: Program-aided language models, 2023. URL <https://arxiv.org/abs/2211.10435>.
- [12] S. Goyal, Z. Ji, A. S. Rawat, A. K. Menon, S. Kumar, and V. Nagarajan. Think before you speak: Training language models with pause tokens, 2024. URL <https://arxiv.org/abs/2310.02226>.
-

-
- [13] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.
 - [14] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models, 2022.
 - [15] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. Large language models cannot self-correct reasoning yet, 2023.
 - [16] A. L. Jones. Scaling scaling laws with board games, 2021. URL <https://arxiv.org/abs/2104.03113>.
 - [17] D. Kahneman. Maps of bounded rationality: Psychology for behavioral economics. *The American Economic Review*, 93(5):1449–1475, 2003.
 - [18] D. Kahneman. *Thinking, fast and slow*. Farrar, Straus and Giroux, New York, first paperback edition edition, 2013.
 - [19] L. Kocsis and C. Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.
 - [20] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo, Y. Wu, B. Neyshabur, G. Gur-Ari, and V. Misra. Solving quantitative reasoning problems with language models, 2022.
 - [21] Y. Li, Z. Lin, S. Zhang, Q. Fu, B. Chen, J.-G. Lou, and W. Chen. Making large language models better reasoners with step-aware verifier, 2023.
 - [22] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step, 2023.
 - [23] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback, 2023.
 - [24] N. McAleese, R. Pokorny, J. F. Cerón Uribe, E. Nitishinskaya, M. Trębacz, and J. Leike. Llm critics help catch llm bugs. *OpenAI*, 2024.
 - [25] OpenAI. Gpt-4 technical report, 2024.
 - [26] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
 - [27] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen. Tool learning with large language models: A survey, 2024. URL <https://arxiv.org/abs/2405.17935>.
 - [28] Y. Qu, T. Zhang, N. Garg, and A. Kumar. Recursive introspection: Teaching foundation models how to self-improve. 2024.
-

-
- [29] N. Sardana and J. Frankle. Beyond chinchilla-optimal: Accounting for inference in language model scaling laws, 2023.
- [30] W. Saunders, C. Yeh, J. Wu, S. Bills, L. Ouyang, J. Ward, and J. Leike. Self-critiquing models for assisting human evaluators, 2022.
- [31] A. Setlur, S. Garg, X. Geng, N. Garg, V. Smith, and A. Kumar. Rl on incorrect synthetic data scales the efficiency of llm math reasoning by eight-fold. *arXiv preprint arXiv:2406.14532*, 2024.
- [32] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [33] A. Sharma, S. Keh, E. Mitchell, C. Finn, K. Arora, and T. Kollar. A critical evaluation of ai feedback for aligning large language models, 2024. URL <https://arxiv.org/abs/2402.12366>.
- [34] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [35] A. Singh, J. D. Co-Reyes, R. Agarwal, A. Anand, P. Patil, X. Garcia, P. J. Liu, J. Harrison, J. Lee, K. Xu, A. Parisi, A. Kumar, A. Alemi, A. Rizkowsky, A. Nova, B. Adlam, B. Bohnet, G. Elsayed, H. Sedghi, I. Mordatch, I. Simpson, I. Gur, J. Snoek, J. Pennington, J. Hron, K. Kenealy, K. Swersky, K. Mahajan, L. Culp, L. Xiao, M. L. Bileschi, N. Constant, R. Novak, R. Liu, T. Warkentin, Y. Qian, Y. Bansal, E. Dyer, B. Neyshabur, J. Sohl-Dickstein, and N. Fiedel. Beyond human data: Scaling self-training for problem-solving with language models, 2024.
- [36] C. Snell, E. Wallace, D. Klein, and S. Levine. Predicting emergent capabilities by finetuning. *Conference on Language Modeling 2024*, 2024.
- [37] K. Stechly, M. Marquez, and S. Kambhampati. Gpt-4 doesn’t know it’s wrong: An analysis of iterative prompting for reasoning problems, 2023.
- [38] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. Second edition, 2018.
- [39] G. Team. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024.
- [40] Y. Tian, B. Peng, L. Song, L. Jin, D. Yu, H. Mi, and D. Yu. Toward self-improvement of llms via imagination, searching, and criticizing, 2024.
- [41] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023. URL <https://arxiv.org/abs/2307.09288>.
- [42] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins. Solving math word problems with process- and outcome-based feedback, 2022.

-
- [43] K. Valmeekam, M. Marquez, and S. Kambhampati. Can large language models really improve by self-critiquing their own plans?, 2023.
 - [44] P. Villalobos and D. Atkinson. Trading off compute in training and inference, 2023. URL <https://epochai.org/blog/trading-off-compute-in-training-and-inference>. Accessed: 2024-07-03.
 - [45] P. Wang, L. Li, Z. Shao, R. X. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations, 2023.
 - [46] R. Wang, E. Zelikman, G. Poesia, Y. Pu, N. Haber, and N. D. Goodman. Hypothesis search: Inductive reasoning with language models, 2024. URL <https://arxiv.org/abs/2309.05660>.
 - [47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023.
 - [48] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models, 2023.
 - [49] Z. Yuan, H. Yuan, C. Li, G. Dong, K. Lu, C. Tan, C. Zhou, and J. Zhou. Scaling relationship on learning mathematical reasoning with large language models, 2023.
 - [50] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022.
 - [51] E. Zelikman, G. Harik, Y. Shao, V. Jayasiri, N. Haber, and N. D. Goodman. Quiet-star: Language models can teach themselves to think before speaking, 2024. URL <https://arxiv.org/abs/2403.09629>.

附录

A. 相关工作

语言模型推理。近年来，语言模型在具有挑战性的数学推理任务上的性能迅速提升 [20, 22, 25, 32, 39]。这些改进可归因于四个主要因素：**1)** 在大量数学相关数据上进行持续预训练 [20, 22, 32, 39]；

2) 通过针对特定推理任务进行强化学习微调来改进语言模型的提议分布 [32, 35, 49, 50]，使模型能够迭代地批判和修正其答案 [4, 8, 23, 30]；**3)** 通过微调验证器使语言模型受益于额外的测试时计算 [6, 7, 10, 22, 40, 42, 45, 48]。本文的工作建立在上述第二和第三条研究路线之上，分析了通过 **1)** 优化语言模型的提议分布和 **2)** 针对验证器进行搜索，能在多大程度上改进测试时计算扩展。

分析测试时计算扩展。琼斯（Jones）[16] 先前研究了将蒙特卡洛树搜索应用于棋盘游戏六贯棋时训练时计算与测试时计算之间的权衡。本文则将分析重点放在全规模语言模型数学推理问题上。维拉洛沃斯（Villalobos）和阿特金森（Atkinson）[44] 的综述工作分析了多个领域中训练与推理之间的权衡。然而，他们关于语言模型的分析大多集中在已知标准答案情况下的测试时计算扩展。相比之下，本文的分析聚焦于标准答案未知的情况。此外，强化学习文献中的许多工作提出了诸如蒙特卡洛树搜索 [19] 等方法，旨在平衡测试时与训练时计算之间的权衡，从而实现一种迭代式自我博弈。本文的研究发现可用于帮助开发类似的算法，使其能够处理开放式自然语言。

通过测试时计算增强语言模型。除验证器与修订之外，大量后续工作提出了使语言模型利用测试时计算进行推理的替代方法。具体而言，Wang 等人 [46] 进行了分层假设搜索，以实现归纳推理能力。多项相关工作提出在测试时为语言模型配备工具，这能显著提升其在下游任务上的性能 [11, 26, 27]。最后，若干工作提出了以无监督方式学习思维令牌的方法 [12, 51]，使模型能更有效地利用采样更长序列所带来的额外测试时计算。尽管本文的分析聚焦于测试时计算可扩展的两种主要机制（即验证器与修订），但本文所采用的许多分析方法（例如根据问题难度进行最优计算扩展）原则上也可应用于任何其他测试时计算扩展方法，本文认为这是未来研究的一个有趣方向。

B. 补充修订结果

本文在图 10 中绘制了使用 PaLM 2-S* 修订模型进行多数选择的额外结果。采用多数选择时，本文观察到与图 7 中验证器选择所呈现的趋势大体相似。

C. 无监督难度分组

本文在没有真实正确性信息的情况下计算难度分组，方法是对每个问题的 2048 个样本的 PRM 最终答案分数取平均，从而获得对应于

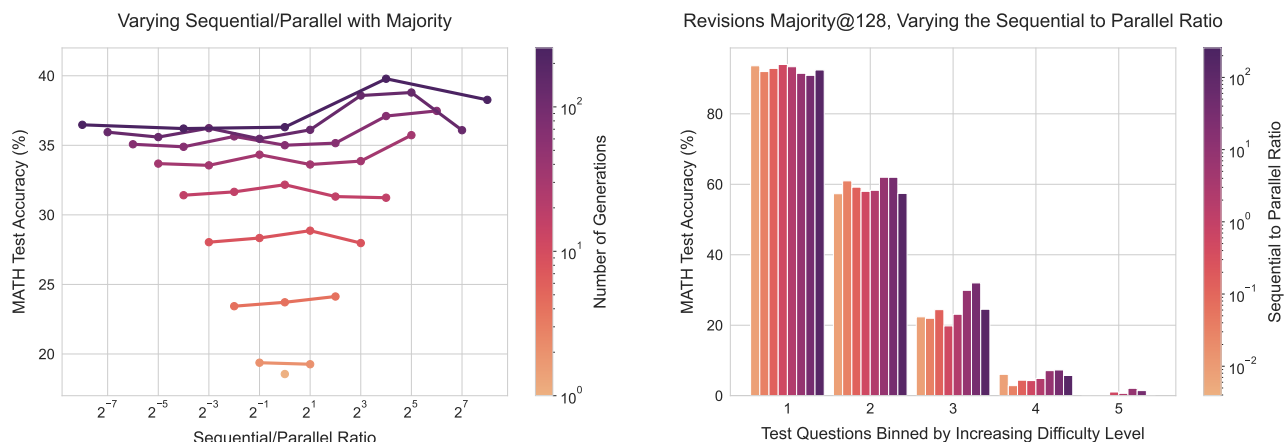


图 10 | 改变分配给顺序样本与并行样本的生成预算比例，使用多数投票而非验证器来选择答案。左图：每条线代表在比例变化时的固定生成预算。本文发现与验证器情况类似，在多数投票情况下，给定预算下存在顺序与并行测试时计算的理想比例。右图：分析不同难度分组下的性能，本文发现较简单的问题对顺序与并行的比例基本不敏感，而较难的问题则存在顺序与并行测试时计算的理想比例。

问题。随后，我们将测试集中每个问题的价值划分为五个五分位数（采用与真实难度分箱相同的程序）。我们将其称为“预测难度”而非“真实难度”。从技术上讲，这一过程极其昂贵，因为它需要生成大量样本。虽然我们在分析中未计入这一成本，但在实际生产环境中，这一成本将构成问题。更高效的方法是微调一个模型，使其能够根据问题直接预测正确性。我们在本工作中未对此进行探索，而是将探索更廉价的难度估计方法留待未来工作。在图 12 中，我们绘制了使用难度分箱的 PRM 搜索（PRM-search）结果，在图 11 中绘制了相应的修订结果。我们发现，在这两种设置下，这些预测分箱展现出与真实分箱相似的趋势。

D. 过程奖励模型训练细节

我们将过程奖励模型（PRM）微调为二元分类器，模型在解的每一步预测一个介于 0 和 1 之间的值。我们使用蒙特卡洛展开得到的软值来训练模型，采用二元交叉熵损失函数（例如 $-(v \log(\hat{v}) + (1 - v) \log(1 - \hat{v}))$ ），其中 v 对应软真值， $\hat{v}$ 为模型预测值）。我们使用 AdamW 优化器对基础模型进行微调，学习率为 $3e-5$ ，批大小为 128，随机失活率为 0.05，Adam betas 为 $(0.9, 0.95)$ 。我们进行早停，在随机留出的验证集上选择验证损失最低的检查点，该验证集包含原始 PRM800k 训练划分中 10% 的问题。

本文在对应少样本提示基础模型生成的每个问题 16 个样本上微调过程奖励模型（PRM）。每一步采用 16 次蒙特卡洛展开，利用相同的基础模型和提示来估计步骤级价值。训练数据中所有未能输出有效、可解析最终答案的样本均被剔除，因为初步实验表明这些样本会损害过程奖励模型的性能。

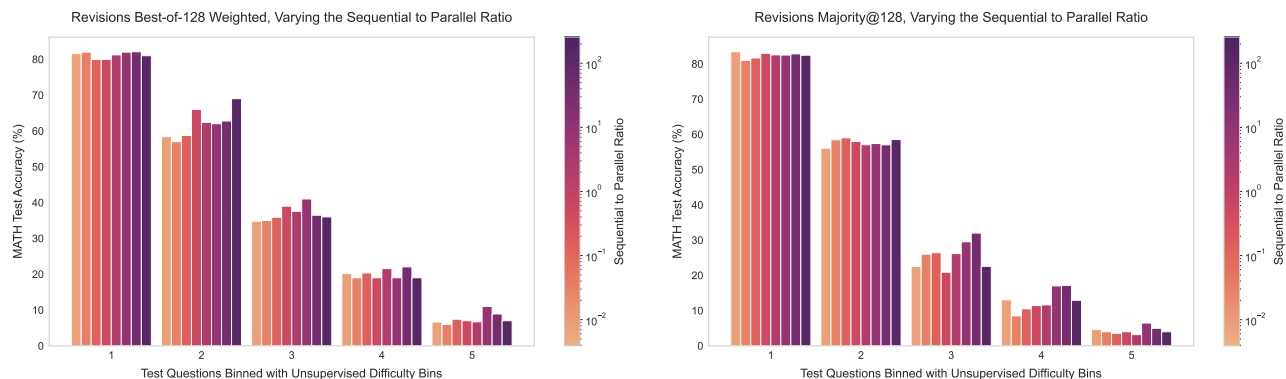


图 11 | 利用我们的 PaLM 2-S*过程奖励模型 (PRM) 计算难度分箱，无需修订版本的真实正确性信息。左侧绘制验证器选择，右侧绘制多数投票选择。我们观察到这些分箱的性能趋势与图 7 中基于真实标签的分箱大体相似。以及 10。

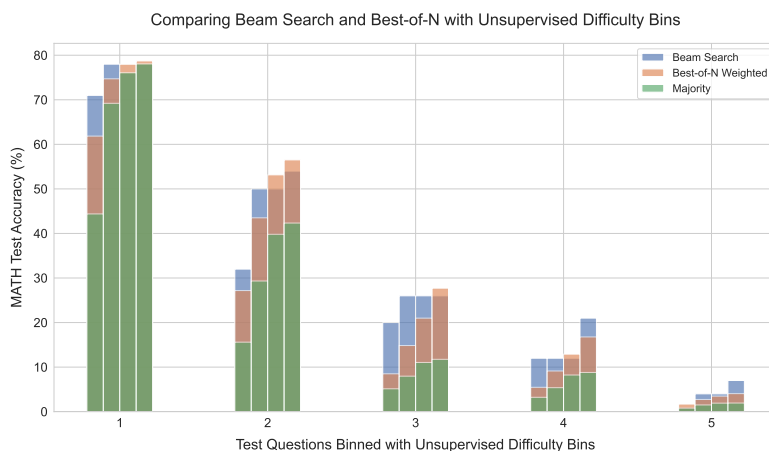


图 12 | 利用我们的 PaLM 2-S*过程奖励模型 (PRM) 计算难度分箱，无需真实正确性信息即可进行 PRM 搜索。我们发现这些分箱的性能趋势与图 3 中使用真实标签的分箱基本一致。

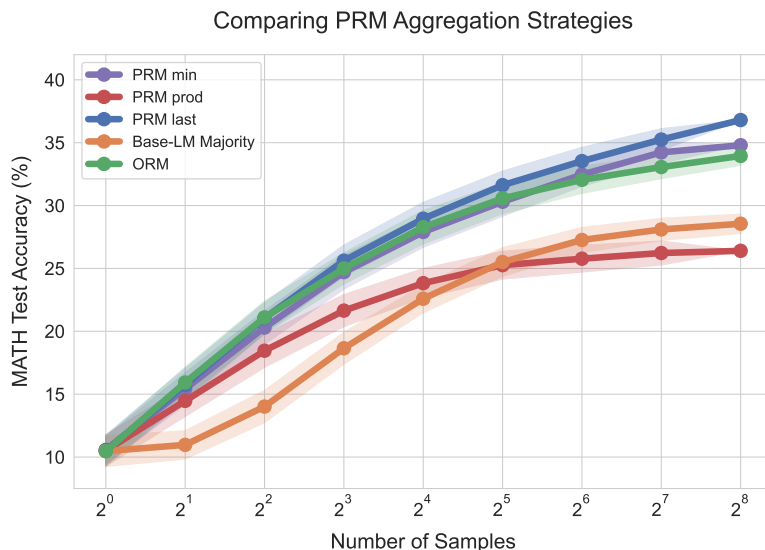


图 13 | 我们比较了聚合逐步 PRM 分数以生成完整解答最终分数的不同方法：“min”表示取所有步骤中的最小分数，“prod”表示取所有步骤正确性概率的乘积，“last”仅使用最后一步的分数。我们发现“last”在所有聚合策略中表现最佳。

在生成样本时，基础模型被提示以换行分隔的逐步格式输出答案，如 Lightman 等人 [22] 所述。随后我们使用简单的换行分割程序将每个答案拆分为步骤。提示详情见附录 G。

E. 比较 PRM 聚合策略

我们比较了聚合逐步 PRM 分数以生成完整解答最终分数的不同方法。具体比较：1) 取所有步骤中的最小分数，如 Lightman 等人 [22] 所述（即“min”）；2) 取所有步骤正确性概率的乘积（即“prod”）；3) 仅使用最后一步的预测（即“last”）。图 13 显示，采用最后一步的方法优于其他两种方法。先前工作 [22, 45] 发现“min”是最佳聚合器。我们认为这种差异源于我们的验证器使用软蒙特卡罗回报标签进行训练，其表现与二元正确性标签截然不同，因此其他聚合策略可能不会产生相同效果。有趣的是，当使用最后一步聚合时，我们实际上将 PRM 用作结果奖励模型（ORM）。然而，我们发现 PRM 优于 ORM，这表明在我们的情况下，逐步 PRM 训练可能主要作为一种表示学习的形式发挥作用，而不仅仅是推理时的工具。未来工作应进一步探索这一推理方向。

F. 比较 PRM 与 ORM

我们使用 PaLM 2-S*基础语言模型训练了 PRM 和 ORM 模型。图 14 显示，PRM 优于 ORM，且 PRM 与 ORM 之间的差距随着样本数量的增加而扩大。

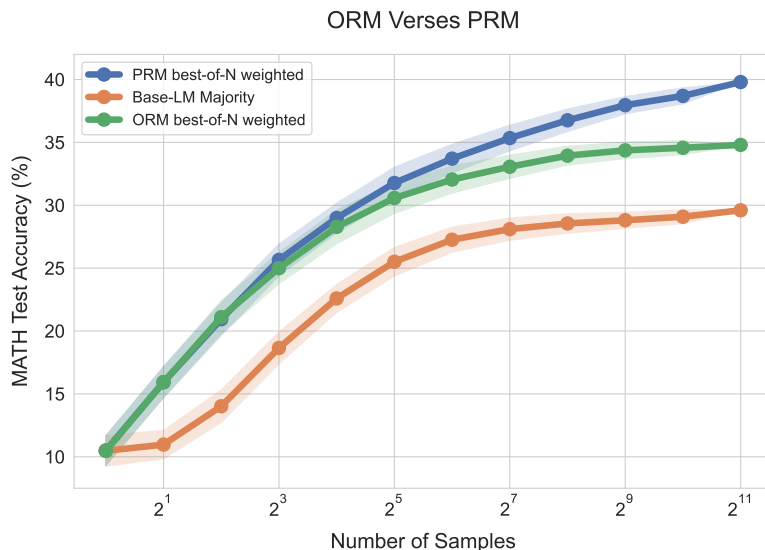


图 14 | 我们在 N 选优评估中比较了从 PaLM 2-S*微调得到的 PRM 和 ORM 模型。我们使用 PaLM 2-S*基础语言模型进行采样输出，并采用少样本提示。我们发现 PRM 在大量样本下显著优于 ORM。

所用样本数量。我们使用过程奖励模型（PRM）的最后一步预测来对答案进行评分，具体方法见附录 E。

G. 提示细节

为了使基础模型能够以逐步格式输出答案，以便应用过程奖励模型（PRM），我们使用了一个由 Lightman 等人发布的 PRM800k 数据集中随机选取的正确答案示例构成的 4 样本提示 [22]。具体来说，我们使用第一阶段训练划分中的答案。这些答案对应于 GPT-4 生成的正确答案示例，其中包含正确的逐步格式。在初步实验中，我们发现这种提示过程产生的结果与 Lewkowycz 等人使用的提示相似 [20]。我们使用该提示来生成 PRM 和修订模型的训练数据。在测试集上对 PRM 进行搜索时，我们也使用该提示。为了评估该提示预测的最终答案，我们使用 Lightman 等人发布的评分函数 [22]。

H. 修订模型微调细节

对于修订模型的微调，我们遵循第 6.1 节中概述的流程。我们首先为每个问题采样 64 个输出。然后过滤掉所有以无效解结尾的答案。对于每个正确答案，我们随后在 0 到 4 之间均匀采样一个数字，表示在训练上下文中包含多少个错误答案。正确答案被用作轨迹中的最后一个答案（我们训练模型生成该答案），错误答案则包含在上下文中。如果采样数字大于 0，我们根据字符级编辑距离度量找到最接近的错误答案，将其作为轨迹中的最后一个错误答案。这里的目标是选择一个错误的

答案与正确答案存在一定相关性，以促进学习。其余错误答案则从可用答案集中随机采样。当采样到的错误答案少于 4 个时，我们将均匀分布的最大值截断至与错误样本数量一致。我们采用此流程为训练数据中的所有问题生成轨迹。随后，在生成轨迹中的正确答案解答上对基础语言模型进行微调。使用 AdamW 优化器，学习率为 $1e-5$ ，批大小为 128，随机失活率为 0.0，Adam 贝塔参数 $(0.9, 0.95)$ 。我们发现，通常在由上述流程生成的轨迹构成的评估集上评估损失，并不能为早停提供良好信号。相反，我们发现在评估损失开始上升之后的许多检查点，模型具备更强的修正能力。这可能是因为微调修正模型后，评估集代表的是离策略数据，与模型自身在策略下生成的轨迹相比，自然处于分布之外。因此，我们在验证集上观察到过拟合之后稍晚的位置选择修正模型检查点。

一、修正模型选择准则

如第 6.1 节所述，为了有效利用我们的修订模型，我们需要部署一个标准，用于在修订轨迹内以及多个并行轨迹之间选择最佳答案。我们采用两种方法：1) 结果奖励模型验证器；2) 多数投票。对于结果奖励模型验证器，我们根据附录 J 中的流程，在修订模型的输出上训练一个结果奖励模型。在推理时，我们使用该验证器选择最佳答案。由于我们有两个聚合轴（每个修订轨迹内和多个轨迹之间），我们部署分层策略，首先在每个修订轨迹内选择最佳答案，然后在轨迹之间聚合这些选定的答案。为了在每个轨迹内选择最佳答案，我们执行 N 选加权聚合，然后选择具有最大 N 选加权答案的最高分解决方案。然后，为了在所有修订链中选择最终答案，我们使用每个修订链的最佳答案进行另一轮 N 选加权选择。第二轮 N 选加权后的答案代表我们的最终答案预测。对于多数投票，我们发现当轨迹长度或轨迹数量太小时，分层聚合会产生问题。问题在于，没有足够的样本，多数投票无法有效选择最佳选项。因此，对于多数投票，我们简单地一次性取所有轨迹中的所有答案，并将它们的多数作为最终答案。我们发现这比分层方法产生更平滑的扩展行为。

J. 修订模型验证器训练

我们发现，在 PaLM 2-S*基础模型输出上微调的过程奖励模型应用于 PaLM 2-S*修订模型的输出时效果不佳（见图 15(a)），可能是由于修订模型的分布偏移。因此，我们训练了一个单独的结果奖励模型验证器，用于我们的 PaLM 2-S*修订模型。我们本可以训练一个过程奖励模型，但由于生成逐步过程奖励模型标签的高成本，我们选择了结果奖励模型。我们针对修订设置对标准结果奖励模型进行了轻微修改，通过在上下文中微调结果奖励模型，使验证器能够访问与修订模型相同的上下文，从而允许

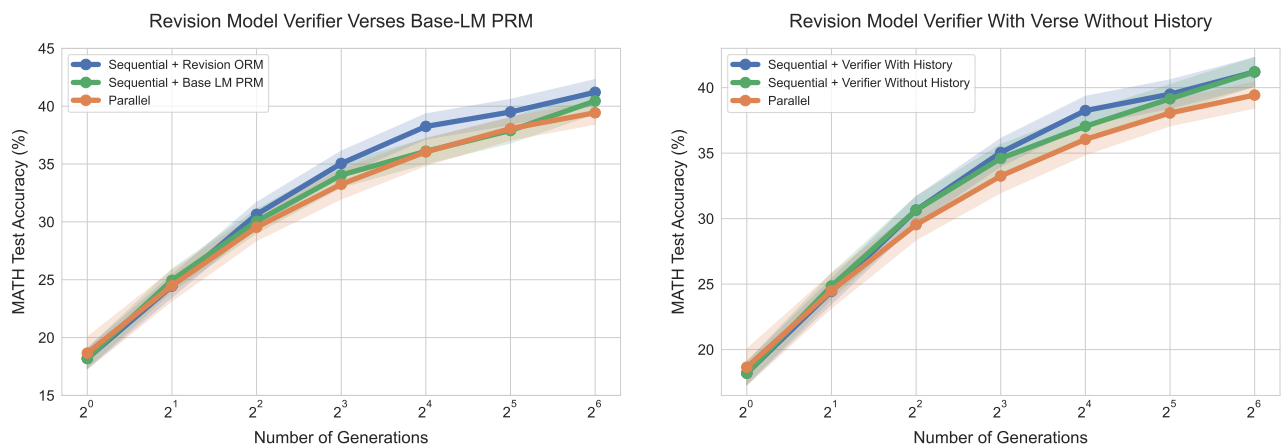


图 15 | 左：我们将基于修订模型输出训练的 ORM 与基于 PaLM 2-S*基础模型输出训练的 PRM 进行比较。可以看到，当应用于修订模型的输出时，针对修订模型适配的 ORM 优于 PRM，这可能是由于与修订模型之间的分布偏移所致。右：我们消融了在修订模型验证器上下文包含先前修订的影响。可以看到，在上下文中包含修订对验证器略有帮助，但两种设置仍优于并行基线。

验证器在评分当前答案时可以看到修订模型先前的答案尝试。所有其他实验细节与训练 PRM 时相同。经验上，我们发现将修订历史包含在上下文中可略微提升性能（见图 15(b)）。此外，即使上下文中不包含修订，顺序修订仍略优于并行，这表明顺序采样带来的改进不仅仅源于验证器的上下文。

K. ReST_{EM} 修订模型实验

我们尝试通过使用简化的强化学习算法 ReST_{EM} [35] 训练模型来进一步优化 PaLM 2-S*修订模型。具体来说，我们为 MATH 训练集中的每个问题生成了 64 条最大长度为 5 的修订轨迹。我们在每条轨迹中第一个正确答案处停止修订模型。利用这些生成的数据，我们在正确答案数据上微调基础语言模型。为了帮助模型学习任务，我们显式平衡了轨迹长度的分布。在图 16 中，我们绘制了该新修订模型在顺序与并行比例变化时的性能。可以看到，额外的顺序修订显著损害了该新模型的性能。我们假设这种退化是由于运行 ReST_{EM} 获得的在线数据加剧了修订数据中的虚假相关性，导致优化后的模型未能学习修订任务。我们认为，采用 Qu 等人 [28] 所采用的更离线数据收集策略可能更有效，并将进一步探索留待未来工作。

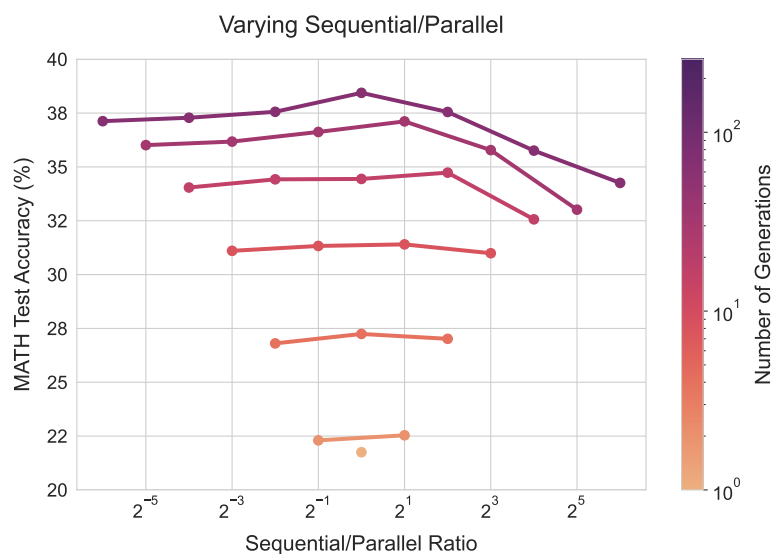


图 16 | 我们的 ReST_{EM} 优化修订模型在顺序与并行比例变化时的性能。我们使用多数投票选择答案。可以看到，该优化修订模型在额外顺序修订下表现出显著的性能下降。

L. 修订模型示例输出

在图 17、18、19、20、21、22 和 23 中，我们展示了修订模型输出的精选示例。

M. PRM 束搜索示例输出

在图 24、25、26、27、28 和 29 中，我们展示了 PRM 束搜索的精选示例。我们在示例中为每一步提供了介于 0 和 1 之间的 PRM 分数。

If $f(x) = \frac{3x-2}{x-2}$, what is the value of $f(-2) + f(-1) + f(0)$? Express your answer as a common fraction.

Attempt 1:

Let's evaluate $f(-2)$.
 We have $f(-2) = \frac{3(-2)-2}{-2-2} = -\frac{4}{-4} = \frac{1}{1} = 1$.
 So $f(-2) = 1$.
 Let's evaluate $f(-1)$.
 We have $f(-1) = \frac{3(-1)-2}{-1-2} = -\frac{5}{-3} = \frac{5}{3}$.
 So $f(-1) = \frac{5}{3}$.
 Let's evaluate $f(0)$.
 We have $f(0) = \frac{3(0)-2}{0-2} = \frac{-2}{-2} = 1$.
 So $f(0) = 1$.
 So we want to compute
 $1 + \frac{5}{3} + 1 = \frac{3}{3} + \frac{5}{3} + \frac{3}{3} = \frac{3+5+3}{3} = \frac{8}{3}$.
 So the answer is $\boxed{\frac{8}{3}}$.
 ##### $\frac{8}{3}$

Attempt 2:

Let's first find $f(-2)$.
 $f(-2) = \frac{3(-2)-2}{-2-2} = \frac{-6-2}{-4} = \frac{-8}{-4} = 2$.
 Now let's find $f(-1)$.
 $f(-1) = \frac{3(-1)-2}{-1-2} = \frac{-3-2}{-3} = \frac{-5}{-3} = \frac{5}{3}$.
 Now let's find $f(0)$.
 $f(0) = \frac{3(0)-2}{0-2} = \frac{-2}{-2} = 1$.
 So we have $f(-2) + f(-1) + f(0) = 2 + \frac{5}{3} + 1 = \frac{7}{3} + \frac{5}{3} + \frac{3}{3} = \frac{7+5+3}{3} = \frac{15}{3} = 5$.
 So the answer is $\boxed{5}$.
 ##### 5

Attempt 3:

Let's plug in -2 , -1 , and 0 for x in $f(x)$.
 We get $f(-2) = \frac{3(-2)-2}{-2-2} = \frac{-8}{-4} = \frac{2}{1} = 2$, $f(-1) = \frac{3(-1)-2}{-1-2} = \frac{-5}{-3} = \frac{5}{3}$, $f(0) = \frac{3(0)-2}{0-2} = \frac{-2}{-2} = 1$.
 So we have $2 + \frac{5}{3} + 1 = \frac{6}{3} + \frac{5}{3} + \frac{3}{3} = \frac{14}{3}$.
 So the answer is $\boxed{\frac{14}{3}}$.
 ##### $\frac{14}{3}$

图 17 | 修订模型示例 1。模型在前两次尝试中最后计算总和时出错，但在第三次尝试中成功并得到正确答案。

In how many ways can 7 people sit around a round table if no two of the 3 people Pierre, Rosa, and Thomas can sit next to each other? (Seating arrangements which are rotations of each other are treated as the same.)

Attempt 1:

```
Let's start by figuring out how many ways there are to seat the three people who
can't sit next to each other.
So we need to seat them in three seats that are not right next to each other.
So there are 4 chairs that they can sit in.
So we have 4 slots to fill with 3 people.
We can do that in 4 choose 3 ways.
That is  $4!/(3! * 1!) = 4$  ways.
Now we need to figure out how many ways there are to seat the remaining 4 people.
So we can seat the first person in any of the seven chairs.
And then we can seat the second person in any of the remaining six chairs.
And so on.
So we can seat the four people in  $7 * 6 * 5 * 4$  ways.
So we need to multiply the number of ways to seat the three people who can't sit
next to each other, and the number of ways to seat the other four people.
That is  $4 * 7 * 6 * 5 * 4 = 840$ .
So the answer is  $\boxed{840}$ .
#### 840
```

Attempt 2:

```
So we have to figure out how many ways there are to seat 7 people around a table
if no two of the 3 people can sit next to each other.
Let's start by seating the 3 people who can't sit next to each other.
We know that they can't sit next to each other, but they can sit on the ends of
the table.
So they can sit in 3 different ways.
Now let's seat the 4 people who can sit anywhere.
We can do that in  $4!$  ways.
Now let's seat the 3 people who can't sit next to each other.
Let's seat them in the 3 spots we found earlier.
So there are 3 different ways to seat them.
So we have  $3 * 4! * 3 = 3 * 24 * 3 = 216$  ways to do it.
So the answer is  $\boxed{216}$ .
#### 216
```

Attempt 3:

```
So we have to find the number of ways to arrange 4 people around a round table
such that 3 of the people can't sit next to each other.
So let's start with the 3 people who can sit next to each other.
There are  $3!$  ways to arrange them.
Now we have to arrange the other 4 people.
There are  $4!$  ways to do that.
So the total number of ways to arrange the 7 people around the table is  $3! * 4! =$ 
 $6 * 24 = 144$ .
So the answer is  $\boxed{144}$ .
#### 144
```

图 18 | 修订模型示例 2。第一次尝试中模型采用了错误的方法，第二次尝试中更接近了，但在接近结束时犯了一个错误。最后一次尝试中它得到了正确答案。

Find the product of $6_8 \cdot 7_8$. Express your answer in base 8.
52_8

Attempt 1:

So we have $6_8 \cdot 7_8$.
Right.
So, we can convert these numbers to base 10.
Right, we have $6_8 = 6_{10}$ and $7_8 = 7_{10}$.
So our equation becomes $6_{10} \cdot 7_{10}$.
So, we have $6_{10} \cdot 7_{10} = 42_{10}$.
So, we need to convert 42_{10} to base 8.
Right.
So, we know that $4_8 = 4_{10}$ and $2_8 = 2_{10}$.
So, we can divide 42_{10} by 8_{10} and get $5 \text{ R } 2$.
So, our answer is 52_8 .
Correct.
52

Attempt 2:

Well, we can start by expanding the numbers in base 10.
Right. So $6_8 = 6$ and $7_8 = 7$.
So we get $6 \cdot 7$.
Right. The product is 42 .
So, we need to convert 42 to base 8.
Right. So, we divide 42 by 8 and get a remainder of 2 and a quotient of 5 .
So, we write 52_8 .
Correct.
52_8

图 19 | 修订模型示例 3。第一次尝试中模型在最终答案的格式上犯了错误；它在第二次尝试中纠正了这一点。

Find the product of $6_8 \cdot 7_8$. Express your answer in base 8.
52_8

Attempt 1:

Let's multiply them together.
We get $6_8 \cdot 7_8 = 42_8$.
So the answer is 42_8 .
42_8

Attempt 2:

We know that $6_8 = 6$, and $7_8 = 7$. So the problem is finding $6 \cdot 7$.
Yep.
 $6 \cdot 7 = 42$.
Correct. So that means $6_8 \cdot 7_8 = 42_8$.
So our answer is 42_8 .
42_8

Attempt 3:

Let's multiply 6_8 by 7_8 in base 8.
 $6 \cdot 7 = 42$ in base 10.
That's correct. So $6_8 \cdot 7_8 = 42_{10} = 40_8 + 2_8$.
We can rewrite this as $40_8 + 2_8 = 40_8 + 10_8 + 10_8 + 8_8 + 8_8 + 8_8 + 8_8 + 1_8$.
And we can rewrite that as $40_8 + 10_8 + 10_8 + 10_8 + 10_8 + 10_8 + 10_8 + 10_8 + 1_8$.
And we can rewrite that as $50_8 + 1_8$.
So $6_8 \cdot 7_8 = 51_8$.
51

Attempt 4:

So $6_8 \cdot 7_8$ is $6 \cdot 7$ in base 8.
That's right.
Let's convert 6 and 7 to decimal. $6 = 6_8 = 6 \cdot 8^0 + 0 \cdot 8^1 = 6$.
Right. And $7 = 7_8 = 7 \cdot 8^0 + 0 \cdot 8^1 = 7$.
Right. So $6 \cdot 7 = 42$.
And 42 in octal is 52_8 .
That's correct.
52_8

图 20 | 修订模型示例 4。前几次尝试中模型未能完成从十进制到八进制的转换。最后一次尝试中它进行了正确的计算。

Convert the point $(0,3)$ in rectangular coordinates to polar coordinates. Enter your answer in the form (r,θ) , where $r > 0$ and $0 \leq \theta < 2\pi$.

Attempt 1:

```
To convert (0, 3) to polar coordinates, we need to find the radius r and the
angle theta.
We know that the x-coordinate of the point is r*cos(theta) and the
y-coordinate of the point is r*sin(theta).
So r*cos(theta) = 0 and r*sin(theta) = 3.
And we know that cos(theta) = 0 when theta = pi/2 and sin(theta) =
3 when theta = 3pi/2.
So we know that (0, 3) corresponds to the point (r, theta) where theta =
pi/2 and r*cos(theta) = 0 and r*sin(theta) = 3.
So r = 3/sin(theta) = 3/sin(pi/2) =
3/1 = 3.
So the polar coordinates of (0, 3) are (3, pi/2).
#### (3, pi/2)
```

Attempt 2:

```
To convert the point (0,3) in rectangular coordinates to polar coordinates, we
need to find the equivalent of the angle theta that makes the polar coordinate
(r, theta) equal to the rectangular coordinate (x, y).
We know that (r, theta) = (0,3) and (x, y) = (0, 3).
So let's set up an equation where r is equal to x and theta is equal to
y.
We have r = x and y = theta.
So we can substitute these into the polar equation r = sqrt(x^2 + y^2).
We get sqrt(x^2 + y^2) = x.
We can square both sides of the equation to get x^2 + y^2 = x^2.
And we can subtract x^2 from both sides of the equation to get y^2 = 0.
And we know that to satisfy y^2 = 0 we must have y = 0.
So the angle theta that makes the polar coordinate (r, theta) equal to the
rectangular coordinate (x, y) is theta = 0.
And the radius r is equal to the x-coordinate, which is x = r = 0.
So the polar coordinate equivalent to (0,3) is (r, theta) = (0,0).
And we know that r > 0 and 0 <= theta < 2pi.
So the polar coordinate equivalent to (0,3) is (r, theta) = (0,0).
#### (0,0)
```

Attempt 3:

```
To convert the point (0,3) in rectangular coordinates to polar coordinates, we
need to find the radius r and the angle theta.
The x-coordinate of the point is 0, so x = r*cos(theta) = 0.
We know that cos(theta) = 0 when theta = pi/2 or theta =
3pi/2.
So we can use either theta = pi/2 or theta = 3pi/2.
The y-coordinate of the point is 3, so y = r*sin(theta) = 3.
We know that sin(theta) = 3/r.
So we can use the equation sin(theta) = 3/r to find r.
We know that sin(pi/2) = 1, so r = 3/1 = 3.
So our polar coordinates are (3, pi/2) or (3, 3pi/2).
And since 0 <= theta < 2pi, we have theta = pi/2.
So our polar coordinates are (3, pi/2).
#### (3, pi/2)
```

图 21 | 修订模型示例 5。在前两次尝试中，模型在将欧几里得坐标转换为极坐标时出现错误。在最后一次尝试中，它没有犯这些错误。

The proper divisors of 12 are 1, 2, 3, 4 and 6. A proper divisor of an integer N is a positive divisor of N that is less than N . What is the sum of the proper divisors of the sum of the proper divisors of 284?

Attempt 1:

```
Ok, let's find the proper divisors of 284 first.  
Right. So the proper divisors of 284 are 1, 2, 4, 71, and 142.  
So the sum of the proper divisors of 284 is 1+2+4+71+142, which is 219.  
Now let's find the proper divisors of 219.  
The proper divisors of 219 are 1, 3, 73, and 219.  
So the sum of the proper divisors of 219 is 1+3+73+219, which is 306.  
So the answer to the question is 306.  
#### 306
```

Attempt 2:

```
So the sum of the proper divisors of 284 is 1+2+4+71+142.  
Ok.  
And the proper divisors of 284 are 1,2,4,71, and 142.  
Right.  
And the sum of those proper divisors is 219.  
So the sum of the proper divisors of 284 is 219.  
Right.  
So the sum of the proper divisors of the sum of the proper divisors of 284 is the  
sum of the proper divisors of 219.  
Right.  
And the proper divisors of 219 are 1, 3, 73, and 219.  
Right.  
So the sum of the proper divisors of 219 is 1+3+73+219=306.  
So the sum of the proper divisors of the sum of the proper divisors of 284 is 306.  
Right.  
So the answer to the question is 306.  
#### 306
```

Attempt 3:

```
Right, so the proper divisors of 284 are 1, 2, 4, 71, and 142.  
So the sum of the proper divisors of 284 is 1+2+4+71+142, which is 220.  
Now let's find the proper divisors of 220.  
The proper divisors of 220 are 1, 2, 4, 5, 10, 11, 20, 22, 44, 55, and 110.  
So the sum of the proper divisors of 220 is 1+2+4+5+10+11+20+22+44+55+110, which  
is 284.  
So the answer to the question is 284.  
#### 284
```

图 22 | 修订模型示例 6。在前两次尝试中，模型在求 284 的真因子之和时出现错误。在第三次尝试中，它正确地计算了这个和。

If $-6 \leq a \leq -2$ and $3 \leq b \leq 5$, what is the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$? Express your answer as a common fraction.

Attempt 26:

So we have $-6 \leq a \leq -2$ and $3 \leq b \leq 5$.
 And we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$.
 So we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $-6 \leq a \leq -2$ and $3 \leq b \leq 5$.
 Ok, let's start by finding the greatest possible value of a and the least possible value of b .
 Ok, so the greatest possible value of a is -2 .
 And the least possible value of b is 3 .
 So we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $a = -2$ and $b = 3$.
 So we want to find the greatest possible value of $\left(-2 + \frac{1}{3}\right)\left(\frac{1}{3} - (-2)\right)$.
 So we want to find the greatest possible value of $\left(-2 + \frac{1}{3}\right)\left(\frac{1}{3} + 2\right)$.
 So we want to find the greatest possible value of $\left(-2 + \frac{1}{3}\right)\left(\frac{5}{3}\right)$.
 So we want to find the greatest possible value of $\left(\frac{-6+1}{3}\right)\left(\frac{5}{3}\right)$.
 So we want to find the greatest possible value of $\left(\frac{-5}{3}\right)\left(\frac{5}{3}\right)$.
 So we want to find the greatest possible value of $\frac{-5}{3} \cdot \frac{5}{3}$.
 So we want to find the greatest possible value of $\frac{-25}{9}$.
 So the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $-6 \leq a \leq -2$ and $3 \leq b \leq 5$ is $\boxed{-\frac{25}{9}}$.
 #### $\frac{-25}{9}$

Attempt 27:

Ok, so we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$.
 So we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $-6 \leq a \leq -2$ and $3 \leq b \leq 5$.
 Ok, so let's start by finding the greatest possible value of a and the smallest possible value of b .
 Ok, so the greatest possible value of a is -2 .
 And the smallest possible value of b is 3 .
 So we want to find the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $a = -2$ and $b = 3$.
 So we want to find the greatest possible value of $\left(-2 + \frac{1}{3}\right)\left(\frac{1}{3} - (-2)\right)$.
 So we want to find the greatest possible value of $\left(-2 + \frac{1}{3}\right)\left(\frac{1}{3} + 2\right)$.
 So we want to find the greatest possible value of $\left(-\frac{5}{3}\right)\left(\frac{7}{3}\right)$.
 So we want to find the greatest possible value of $-\frac{5}{3} \cdot \frac{7}{3}$.
 So $-\frac{5}{3} \cdot \frac{7}{3} = \frac{-35}{9}$.
 So the greatest possible value of $\left(a + \frac{1}{b}\right)\left(\frac{1}{b} - a\right)$, where $-6 \leq a \leq -2$ and $3 \leq b \leq 5$ is $\boxed{-\frac{35}{9}}$.
 #### $-\frac{35}{9}$

图 23 | 修订模型示例 7。在第一次尝试中，模型错误地计算了 $\frac{1}{3} + 2$ 。在第二次尝试中，它纠正了这个错误。

The sum of the first n terms in the infinite geometric sequence $\left(\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots\right)$ is $\frac{255}{512}$. Find n .

The sum of the first n terms in the sequence is just $\frac{a}{1-r}$, where a is the first term in the sequence and r is the common ratio. [score = 0.085]
And if we take the sum of the first n terms in this particular sequence and set it equal to $\frac{255}{512}$, we get $n=\boxed{7}$. [score = 0.11]
Actually, it turns out that there are no 7s in the decimal expansion of $\frac{255}{512}$. [score = 0.10]
Right, so we can set $\frac{a}{1-\frac{1}{4}\{1-\frac{1}{2^n}\}}\{1-\frac{1}{2}\} = \frac{255}{512}$, and solve for n . [score = 0.10]
So we get $n=7$. [score = 0.10]
That gives $1-\left(\frac{1}{2}\right)^n = \frac{255}{256}$. [score = 0.15]
So, $1-\left(\frac{1}{2}\right)^n = \frac{255}{256}$. [score = 0.15]
7 [score = 0.30]

Figure 24 | PRM beam search example 1.

The sum of the first n terms in the infinite geometric sequence $\left(\frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots\right)$ is $\frac{255}{512}$. Find n .

Since the common ratio in each term is $\frac{1}{2}$, the sum of the first n terms of this geometric sequence is $\frac{a_1(1-r^n)}{1-r}$, where a_1 is the first term and r is the common ratio. [score = 0.09]
So, the sum of the first n terms of this geometric sequence is $\frac{\frac{1}{4}\left(1-\left(\frac{1}{2}\right)^n\right)}{1-\frac{1}{2}}$. [score = 0.09]
So, the sum of the first n terms of this geometric sequence is $\frac{1}{4}\left(1-\left(\frac{1}{2}\right)^n\right)$. [score = 0.11]
Right. And we want $s_n = \frac{255}{512}$. So we have $\frac{1}{4}\left(1-\left(\frac{1}{2}\right)^n\right) = \frac{255}{512}$. [score = 0.14]
We can multiply both sides by 4. [score = 0.10]
We have $1-\left(\frac{1}{2}\right)^n = \frac{255}{128}$. [score = 0.14]
So $\left(\frac{1}{2}\right)^n = 1 - \frac{255}{128}$. [score = 0.16]
So, $\left(\frac{1}{2}\right)^n = \frac{1}{128}$. [score = 0.30]
And then, $2^{-n} = 2^{-8}$. [score = 0.80]
So $-n = -8$. [score = 0.78]
Right. And since the bases are the same, the exponents must be equal. [score = 0.83]
8 [score = 0.69]

Figure 25 | PRM beam search example 2.

$441+2(21)(19)+361=x$. Solve for x .

We know that $a^2+2ab+b^2=(a+b)^2$. [score = 0.33]
So $441+2\cdot 21\cdot 19+361=(21+19)^2$. [score = 0.49]
So we get $(21+19)^2=x$. [score = 0.56]
So we get $x=40^2=1600$. [score = 0.90]
So we get $x=1600$. [score = 0.94]
And we know that $x=1600$. [score = 0.95]
That's our answer. [score = 0.93]
So $441+2(21)(19)+361=1600$. [score = 0.93]
So $x=1600$. [score = 0.93]
Right. So $441+2(21)(19)+361=1600$. [score = 0.94]
1600 [score = 0.91]

Figure 26 | PRM beam search example 3.

Determine the coordinates of the point P on the line $y=-x+6$ such that P is equidistant from the points $A(10,-10)$ and $O(0,0)$ (that is, so that $PA=PO$). Express your answer as an ordered pair (a,b) .

So we're given the line $y=-x+6$ and we need to find the point on that line where the line segment from that point to the point with coordinates $(10,-10)$ is the same length as the line segment from the point to the origin. [score = 0.01]
Right. So one way to do that is to use the distance formula. Let's find the distance between P and $(10,-10)$ and the distance between P and the origin. [score = 0.01]
Right. So the distance between P and $(10,-10)$ will be the square root of $(x_P-10)^2+(y_P+10)^2$. Similarly the distance between P and the origin will be the square root of $(x_P)^2+(y_P)^2$. [score = 0.01]
So the two distances must be equal. So $(x_P-10)^2+(y_P+10)^2=(x_P)^2+(y_P)^2$. Expanding the left side gives $x_P^2-20x_P+100+y_P^2+20y_P+100=x_P^2+y_P^2$. [score = 0.02]
So now we can subtract the x_P^2 from both sides to get $-20x_P+20y_P+200=0$. [score = 0.02]
But we know that $y_P=-x_P+6$. So we can substitute that value in to get $-20x_P+20(-x_P+6)+200=0$. [score = 0.03]
Then we can distribute the 20 to get $-20x_P-20x_P+120+200=0$. Now we can combine the terms to get $-40x_P+320=0$. Then we can subtract 320 from both sides to get $-40x_P=-320$. Divide both sides by -40 to get $x_P=8$. [score = 0.12]
We know that $y_P=-x_P+6$ and $x_P=8$, so if we substitute 8 for x_P we get $y_P=-8+6$. [score = 0.22]
Right. Plugging in the values for x_P and y_P we found into the equation $y=-x+6$, we get $y=-8+6$, which simplifies to $y=-2$. [score = 0.24]
So the point $(8,-2)$ is equidistant from the given points. [score = 0.36]
That's correct. [score = 0.35]
(8,-2) [score = 0.39]

Figure 27 | PRM beam search example 4.

Determine the coordinates of the point P on the line $y = -x + 6$ such that P is equidistant from the points $A(10, -10)$ and $O(0, 0)$ (that is, so that $PA = PO$). Express your answer as an ordered pair (a, b) .

So we're given the line $y = -x + 6$ and we need to find the point on that line where the line segment from that point to the point with coordinates $(10, -10)$ is the same length as the line segment from the point to the origin. [score = 0.01]

Right. So one way to do that is to use the distance formula. Let's find the distance between P and $(10, -10)$ and the distance between P and the origin. [score = 0.01]

Right. So the distance between P and $(10, -10)$ will be the square root of $(x_P - 10)^2 + (y_P + 10)^2$. Similarly the distance between P and the origin will be the square root of $(x_P)^2 + (y_P)^2$. [score = 0.01]

So the two distances must be equal. So $(x_P - 10)^2 + (y_P + 10)^2 = (x_P)^2 + (y_P)^2$. Expanding the left side gives $x_P^2 - 20x_P + 100 + y_P^2 + 20y_P + 100 = x_P^2 + y_P^2$. [score = 0.02]

So now we can subtract the x_P^2 from both sides to get $-20x_P + 20y_P + 200 = 0$. [score = 0.02]

But we know that $y_P = -x_P + 6$. So we can substitute that value in to get $-20x_P + 20(-x_P + 6) + 200 = 0$. [score = 0.03]

Then we can distribute the 20 to get $-20x_P - 20x_P + 120 + 200 = 0$. Now we can combine the terms to get $-40x_P + 320 = 0$. Then we can subtract 320 from both sides to get $-40x_P = -320$. Divide both sides by -40 to get $x_P = 8$. [score = 0.12]

So now we know that $x_P = 8$. Then we can substitute that value into the equation $y_P = -x_P + 6$ to find y_P . That gives $y_P = -8 + 6$ which simplifies to $y_P = -2$. [score = 0.30]

3, 3 [score = 0.12]

Figure 28 | PRM beam search example 5.

If 4 daps = 7 yaps, and 5 yaps = 3 baps, how many daps equal 42 baps?

So 4 daps is equal to 7 yaps, so that means that 1 dap is equal to $7/4$ yaps or 1.75 yaps. [score = 0.21]

And 5 yaps is equal to 3 baps, so that means that 1 yap is equal to $3/5$ baps or 0.6 baps. [score = 0.32]

And we can plug these values in to find the answer. [score = 0.31]

So 42 baps is equal to $42 / 0.6$ yaps or 70 yaps. [score = 0.60]

And 70 yaps is equal to $70 / 1.75$ daps or 40 daps. [score = 0.87]

So the answer is $\boxed{40}$. [score = 0.92]

And we're done! [score = 0.91]

And that is our final answer. [score = 0.90]

[score = 0.91]

So 40 daps is equal to 42 baps. [score = 0.90]

And we're done! [score = 0.91]

And that is our final answer. [score = 0.90]

[score = 0.64]

[score = 0.78]

40 [score = 0.83]

Figure 29 | PRM beam search example 6.